LAPORAN TUGAS BESAR 1 IF3070 DASAR INTELIGENSI ARTIFISIAL

Optimasi Solusi Penataan Posisi dan Orientasi Kendaraan pada Kapal dengan Local Search



Disusun oleh G13 ‘inggris bng’

Daniel Arrigo Manurung	18224031
Kenneth Moses Saragih	18224093
Laurensius Dani Rendragraha	18224107

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI **1**
BAB I DESKRIPSI PERSOALAN **3**
1.1. Latar Belakang **3**
1.2. Perumusan Masalah **3**
1.3. Tujuan Pengerjaan **3**
1.4. Ruang Lingkup **4**
1.5. Daftar Variabel **4**
1.6. Ketentuan Representasi **5**
1.7. Fungsi Objektif **6**
BAB II PEMBAHASAN **7**
2.1. Kelas State **7**
2.2. Implementasi Algoritma Hill Climbing Steepest Ascent **12**
2.2.1. Gambaran Umum 12
2.2.2. Input dan Parameter 13
2.2.3. Initial State 13
2.2.4. Objective Function 14
2.2.5. Neighbor Generation 14
2.2.6. Aturan Pemilihan Neighbor 15
2.2.7. Best State and Stagnation 15
2.2.8. Kriteria Penghentian 16
2.2.9. History dan Visualisasi 16
2.2.10. Output 17
2.3. Implementasi Algoritma Genetic Algorithm **17**
2.3.1. Gambaran Umum 17
2.3.2. Input, Parameter, dan Populasi Awal 18
2.3.3. Evaluasi Fitness 19
2.3.4. Roulette Wheel Selection 19
2.3.5. One-Point Crossover dan Reject + Retry 21
2.3.6. Mutasi dengan Random Neighbor 22
2.3.7. Elitism dan Pergantian Generasi 23
2.3.8. Kondisi Berhenti, History, dan Output 24
2.3.9. Visualisasi dan Pengujian 25
2.4. Implementasi Algoritma Simulated Annealing **26**
2.4.1. Gambaran Umum 26
2.4.2. Input dan Parameter 26
2.4.3. Initial State 27
2.4.4. Objective Function 28
2.4.5. Neighbor Generation 28
2.4.6. Acceptance Probability 28

2.4.7. Best State and Stagnation....29
2.4.8. Cooling Schedule and Stopping Criteria....30
2.4.9. History dan Visualisasi....31
2.4.10. Output....33
2.5. Eksperimen Algoritma Hill Climbing Steepest Ascent....34
2.6. Eksperimen Algoritma Genetic Algorithm....38
2.6.1. Tujuan dan konfigurasi eksperimen....38
2.6.2. Rekapitulasi seluruh percobaan....39
2.6.3. Pengaruh jumlah generasi....40
2.6.4. Pengaruh ukuran populasi....41
2.6.5. Kualitas solusi dan batas optimum....42
2.6.6. Visualisasi state dan perkembangan objective....43
2.6.7. Ringkasan hasil eksperimen....62
2.7. Eksperimen Algoritma Simulated Annealing....62
2.7.1. Visualisasi Hasil Eksperimen....64
2.8. Analisis Hasil Eksperimen....70
BAB III KESIMPULAN DAN SARAN....73
3.1. Simpulan....73
3.2. Saran....74
BAB IV Bonus....76
3.3. Bonus No. 1 Penambahan Objective Function....76
3.4. Bonus No. 2 Implementasi Hill-Climbing Tambahan....76
BAB V PEMBAGIAN TUGAS....86
BAB VI REFERENSI....87
LAMPIRAN....88
1. Form Penggunaan Generative AI....88

BAB I DESKRIPSI PERSOALAN

1.1. Latar Belakang

PT Intim Line mengoperasikan kapal kargo pengangkut kendaraan. Pada setiap keberangkatan, perusahaan menerima kendaraan dengan dimensi, berat, tarif pengiriman, dan perkiraan waktu kedatangan (Estimated Time of Arrival atau ETA) yang berbeda. Penataan secara manual dapat menyebabkan ruang geladak kurang termanfaatkan dan total pendapatan pengiriman belum maksimal.

Tugas besar ini membahas pemilihan kendaraan beserta posisi dan orientasinya pada geladak kapal. Tujuannya adalah memaksimalkan total tarif pengiriman dengan memenuhi batasan dimensi geladak, larangan tumpang tindih, dan kapasitas berat kapal. Pencarian solusi dilakukan menggunakan salah satu varian Hill Climbing, Simulated Annealing, dan Genetic Algorithm.

1.2. Perumusan Masalah

Berdasarkan latar belakang tersebut, rumusan masalah pada tugas besar ini adalah:

1. Bagaimana merepresentasikan konfigurasi pemuatan dan membentuk keadaan awal secara acak?
2. Bagaimana mencari susunan kendaraan yang memaksimalkan total tarif pengiriman dengan memenuhi seluruh batasan?
3. Bagaimana perbandingan kualitas solusi, waktu pencarian, dan konsistensi hasil ketiga algoritma?

1.3. Tujuan Pengerjaan

Berdasarkan perumusan masalah tersebut, tujuan pengerjaan pada tugas besar ini adalah:

1. Merancang representasi solusi, pembentukan keadaan awal, dan fungsi objektif.
2. Mengimplementasikan Hill Climbing, Simulated Annealing, dan Genetic Algorithm menggunakan Python.
3. Memvisualisasikan keadaan awal, keadaan akhir, dan hasil eksperimen.
4. Membandingkan performa ketiga algoritma.

1.4.Ruang Lingkup

Ruang lingkup dan batas persoalan adalah sebagai berikut:

1. Persoalan melibatkan satu kapal dengan dimensi geladak dan kapasitas berat tertentu.
2. Geladak dan kendaraan berbentuk persegi atau persegi panjang pada grid dua dimensi dengan koordinat bilangan bulat.
3. Kendaraan dapat dirotasi 90 derajat dan tidak seluruh kendaraan wajib diangkut.
4. Kendaraan yang dimuat harus sepenuhnya berada di dalam geladak, tidak bertumpang tindih, dan tidak menyebabkan kelebihan kapasitas berat.
5. Keadaan awal dibangkitkan secara acak.
6. Fungsi objektif utama memaksimalkan total tarif pengiriman kendaraan yang dimuat. Sebagai fitur bonus, tersedia mode ETA priority yang memaksimalkan total poin prioritas berdasarkan ETA kendaraan. Kedua mode digunakan secara terpisah.

1.5.Daftar Variabel

Entitas	Atribut	Keterangan
Kapal	Panjang dan lebar	Dimensi geladak dalam satuan grid.
Kapal	Kapasitas maksimum	Batas total berat kendaraan dalam satuan unit.
Kendaraan	ID	Identitas unik kendaraan.
Kendaraan	Panjang dan lebar	Dimensi kendaraan dalam satuan grid.
Kendaraan	Berat	Berat kendaraan dalam satuan unit.
Kendaraan	*ShippingFee*	Tarif pengiriman dalam juta rupiah.
Kendaraan	ETA	Perkiraan waktu kedatangan dalam sisa hari.

Penempatan	Koordinat	Posisi kendaraan pada geladak.
Penempatan	Orientasi	Orientasi asli atau hasil rotasi 90 derajat.

1.6.Ketentuan Representasi

Konfigurasi terdiri atas data kapal dan daftar seluruh kendaraan. Suatu state direpresentasikan sebagai daftar kendaraan yang dimuat, beserta ID, koordinat, dan dimensi efektifnya. Kendaraan yang tidak tercantum dalam daftar tersebut dinyatakan berada di luar kapal. Orientasi direpresentasikan melalui pertukaran panjang dan lebar kendaraan.

Koordinat menggunakan titik asal di sudut kiri atas geladak, dengan sumbu (x) mengikuti lebar dan sumbu (y) mengikuti panjang kapal. Setiap penempatan diperiksa agar berada dalam batas geladak, tidak bertumpang tindih, dan memenuhi kapasitas berat.

Pada aplikasi, konfigurasi bawaan kapal adalah panjang 20 grid, lebar 10 grid, dan kapasitas 10.000 unit, dengan daftar kendaraan awal kosong. Pengguna dapat menambahkan kendaraan secara manual atau melalui generator acak.

Keadaan awal pencarian dibentuk dengan mengacak urutan kendaraan, orientasi, dan koordinat penempatannya. Kendaraan dimuat apabila penempatannya valid, sedangkan kendaraan yang gagal ditempatkan dalam batas percobaan, tetap berada di luar kapal. Mode standar mengacak seluruh posisi, sedangkan mode tambahan dapat mempertahankan posisi awal yang diisi pengguna.

Perubahan state yang diperbolehkan meliputi:

1. Menukar dua kendaraan, termasuk kendaraan di dalam dan di luar kapal.
2. Memindahkan kendaraan ke koordinat lain, memasukkannya, atau mengeluarkannya dari kapal.
3. Mengubah orientasi kendaraan sebesar 90 derajat.

Keadaan akhir menampilkan susunan kendaraan di geladak, kendaraan yang tidak dimuat, dan nilai objektifnya.

1.7.Fungsi Objektif

Fungsi objektif utama menggunakan total tarif pengiriman kendaraan yang dimuat, dengan ketentuan sebagai berikut:

1. Total Tarif Pengiriman: Nilai suatu solusi diperoleh dari penjumlahan *ShippingFee* seluruh kendaraan yang diangkut. Semakin besar total tarif pengiriman, semakin baik solusi tersebut.
2. Validitas Penempatan: Kendaraan yang diangkut harus berada sepenuhnya di dalam geladak dan tidak saling bertumpang tindih. Konfigurasi yang melanggar ketentuan ini dinyatakan tidak valid.
3. Batas Kapasitas Kapal: Total berat kendaraan yang diangkut tidak boleh melebihi kapasitas maksimum kapal. Konfigurasi yang melebihi kapasitas tidak diterima sebagai solusi valid.

Pada mode shipping fee, fungsi objektif bertujuan memaksimalkan total tarif pengiriman dengan tetap memenuhi seluruh batasan. Berdasarkan aspek-aspek tersebut, fungsi objektif bertujuan untuk memaksimalkan total tarif pengiriman dengan tetap memenuhi seluruh batasan. Pemeriksaan batasan dilakukan melalui validasi, bukan penambahan penalti pada nilai objektif. Oleh karena itu, semakin besar nilai objektif suatu solusi yang valid, semakin baik kualitasnya. Sebagai fitur bonus, mode ETA priority menilai setiap kendaraan yang dimuat dengan poin sebesar ETA maksimum seluruh kendaraan pada konfigurasi ditambah satu, kemudian dikurangi ETA kendaraan tersebut. Kendaraan dengan ETA lebih kecil memperoleh poin lebih tinggi. Mode ini memaksimalkan jumlah poin prioritas kendaraan yang dimuat dan tidak menggunakan shipping fee. Kedua mode tetap memakai batasan penempatan dan kapasitas yang sama.

BAB II PEMBAHASAN

2.1. Kelas State

State merepresentasikan satu solusi penempatan kendaraan pada kapal. Representasi ini menjadi dasar bersama untuk Hill Climbing, Simulated Annealing, dan Genetic Algorithm, sehingga ketiganya menggunakan aturan validitas dan penilaian yang sama.

Pada implementasi, tidak terdapat kelas tunggal bernama 'State'. Data dipisahkan menjadi 'ConfigModel' untuk mendefinisikan persoalan dan daftar 'VehiclePlacement' untuk menyimpan arrangement yang sedang dievaluasi. Pemisahan ini memungkinkan posisi serta kendaraan yang dimuat berubah selama pencarian tanpa mengubah data kapal dan atribut kendaraan pada konfigurasi.

2.1.1. Konfigurasi Kapal dan Kendaraan

```
class Ship(BaseModel):
    length: int = Field(gt=0)
    width: int = Field(gt=0)
    max_weight: int = Field(gt=0)


class Vehicle(BaseModel):
    id: UUID = Field(default_factory=uuid4)
    x: int | None = Field(default=None, ge=0)
    y: int | None = Field(default=None, ge=0)
    length: int = Field(gt=0)
    width: int = Field(gt=0)
    weight: int = Field(gt=0)
    eta: int = Field(ge=0)
    shipping_fee: int = Field(ge=0)


class ConfigModel(BaseModel):
    ship: Ship
    vehicles: list[Vehicle]
    preserve_positions: bool = False
    objective_mode: Literal["shipping_fee",
"eta_priority"] = "shipping_fee"
```

Kelas 'Ship' menyimpan panjang, lebar, dan kapasitas berat kapal. Kelas 'Vehicle' menyimpan ID unik, dimensi, berat, ETA, shipping fee, serta

koordinat input opsional. 'ConfigModel' menggabungkan kapal dan seluruh kendaraan, termasuk kendaraan yang nantinya tidak dimuat.

'preserve_positions' menentukan mode inisialisasi. Nilai 'False' berarti posisi awal diacak, sedangkan 'True' mempertahankan pasangan koordinat input yang sudah terisi dan mengacak penempatan kendaraan yang belum mempunyai koordinat. Posisi input bukan batasan permanen selama optimasi. Panjang, lebar, dan berat harus positif, sedangkan ETA serta shipping fee tidak boleh negatif.

Atribut objective_mode menentukan fungsi penilaian yang digunakan seluruh algoritma. Nilai shipping_fee merupakan mode bawaan untuk objective wajib, sedangkan eta_priority mengaktifkan objective bonus berbasis ETA.

2.1.2. Representasi Arrangement

```
class VehiclePlacement(BaseModel):
    vehicle_id: UUID
    x: int = Field(ge=0)
    y: int = Field(ge=0)
    length: int = Field(gt=0)
    width: int = Field(gt=0)
```

Satu arrangement direpresentasikan sebagai 'list[VehiclePlacement]', dengan setiap elemen menyimpan satu kendaraan yang dimuat. 'vehicle_id' merujuk ID pada konfigurasi, 'x' dan 'y' menyimpan koordinat sudut kiri atas, sedangkan 'length' dan 'width' menyimpan dimensi pada orientasi saat ini. Rotasi dilakukan dengan menukar panjang dan lebar.

Titik '(0, 0)' berada di kiri atas geladak. Sumbu X mengikuti lebar dan sumbu Y mengikuti panjang. Kendaraan yang ID-nya tidak terdapat pada daftar placement dianggap berada di luar kapal, data kendaraan tersebut tetap tersimpan pada konfigurasi. Sebagai contoh, jika konfigurasi berisi V1, V2, dan V3 tetapi arrangement hanya menyimpan V1 dan V3, maka V2 tidak dimuat.

Koordinat pada 'Vehicle' merupakan input awal, sedangkan posisi aktual hasil inisialisasi atau pencarian berada pada 'VehiclePlacement'. Karena itu, tampilan initial/final state menggunakan placement aktual dan bukan sekadar membaca ulang koordinat input. Pada GA, satu arrangement menjadi satu individu dan sekumpulan individu membentuk populasi.

2.1.3. Validasi State

Fungsi 'validate_arrangement()' mengembalikan daftar kesalahan, jika daftar kosong berarti state valid. Pemeriksaan mencakup ID yang dikenal dan tidak berulang, dimensi asli atau hasil rotasi, kendaraan sepenuhnya di dalam geladak, tidak overlap, serta total berat tidak melebihi kapasitas kapal.

```
def placements_overlap(first: VehiclePlacement,
second: VehiclePlacement) -> bool:
    """Touching edges/corners are not overlap."""
    return (
        first.x < second.x + second.width
        and second.x < first.x + first.width
        and first.y < second.y + second.length
        and second.y < first.y + first.length
    )
```

Fungsi 'placements_overlap()' memeriksa perpotongan dua persegi panjang pada sumbu X dan Y. Penggunaan perbandingan ketat membuat kendaraan yang hanya bersentuhan pada tepi atau sudut tidak dianggap overlap. Untuk posisi kustom, 'validate_initial_arrangement()' juga memastikan X dan Y diisi berpasangan atau sama-sama dikosongkan.

2.1.4. Pembentukan Initial State

State awal dibentuk melalui 'initialize_arrangement()' menggunakan objek RNG. Pada mode random, fungsi mengacak urutan kendaraan, memeriksa sisa kapasitas berat, lalu mencoba orientasi dan koordinat yang memungkinkan. Berikut bagian yang mencoba placement kendaraan.

```
for _ in range(max_attempts_per_vehicle):
    check_cancellation(should_cancel)
    length, width = rng.choice(orientations)
    candidate = VehiclePlacement(
        vehicle_id=vehicle.id,
        x=rng.randint(0, config.ship.width -
width),
        y=rng.randint(0, config.ship.length -
```

```
length),
        length=length,
        width=width,
    )
    if not any(placements_overlap(candidate, other)
for other in placements):
        placements.append(candidate)
        total_weight += vehicle.weight
        break
```

Kandidat hanya dimasukkan jika tidak overlap dengan kendaraan yang sudah ditempatkan. Orientasi dan rentang koordinat sebelumnya dibatasi agar kendaraan berada di dalam geladak, sedangkan kapasitas berat diperiksa sebelum percobaan. Batas default adalah 100 percobaan per kendaraan. Kendaraan yang belum berhasil dimuat tetap berada di luar, tetapi kegagalan sampling ini tidak membuktikan bahwa tidak ada posisi valid sama sekali.

Objek 'Random(seed)' digunakan agar inisialisasi dapat direproduksi. Masing-masing algoritma meneruskan RNG tersebut ke proses pencariannya. Perbedaan jumlah pemanggilan random selama algoritma berjalan dapat menghasilkan jalur pencarian yang berbeda meskipun seed-nya sama.

2.1.5. Objective Function

Fungsi calculate_objective() menghitung nilai state berdasarkan objective_mode pada konfigurasi. Mode shipping_fee menjumlahkan tarif pengiriman kendaraan yang dimuat, sedangkan mode eta_priority menjumlahkan poin prioritas berdasarkan ETA. Pada kedua mode, nilai yang lebih besar menunjukkan solusi yang lebih baik. Dengan fungsi berikut:

Shipping fee:

$$f(S) = \Sigma \text{ shipping_fee}(v)$$

ETA priority:

$$f(S) = \Sigma \ (ETA_max + 1 - ETA(v))$$

untuk setiap kendaraan v pada state S. Bagian perhitungannya pada 'calculate_objective()' adalah sebagai berikut.

```
maximum_eta = max((vehicle.eta for vehicle in
config.vehicles), default=0)
```

```
loaded_ids: set[UUID] = set()
total = 0

for placement in placements:
    vehicle = vehicles.get(placement.vehicle_id)
    if vehicle is None:
        raise ValueError(f"Unknown vehicle
{placement.vehicle_id}.")
    if vehicle.id in loaded_ids:
        raise ValueError(f"Vehicle {vehicle.id} is
placed more than once.")

    loaded_ids.add(vehicle.id)
    total += (
        maximum_eta + 1 - vehicle.eta
        if config.objective_mode == "eta_priority"
        else vehicle.shipping_fee
    )

return total
```

Fungsi mengambil kendaraan berdasarkan ID, memastikan kendaraan tidak dihitung berulang, lalu menjumlahkan kontribusinya sesuai mode objective. Pada mode ETA priority, shipping fee diabaikan dan kendaraan dengan ETA lebih kecil mendapat poin lebih besar. Dalam GA, nilai objective langsung digunakan sebagai fitness individu.

Perhitungan objective tidak menggantikan validasi fisik. State harus tetap memenuhi aturan pada Subbab 2.1.3. Objective tidak menggunakan penalti overlap atau kelebihan berat sebagai pengganti validasi. ETA digunakan pada mode bonus eta_priority, sedangkan mode wajib shipping_fee tetap hanya menggunakan tarif pengiriman.

2.1.6. Operasi Neighbor Bersama

Perubahan arrangement disediakan pada 'neighbors.py'. Operasi menghasilkan kandidat valid tanpa mengubah state asal. Jika perubahan tidak memungkinkan atau tidak menghasilkan perubahan state, fungsi mengembalikan 'None'.

Operasi	Perubahan state
move_vehicle()	Memindahkan kendaraan ke koordinat lain dimana perpindahan ke/dari luar mencakup unload/load

swap_vehicles()	Menukar posisi dua kendaraan yang dimuat, atau mengganti kendaraan di dalam dengan kendaraan di luar
rotate_vehicle()	Menukar panjang dan lebar pada anchor yang sama
random_neighbor()	Mencoba kategori move, swap, atau rotate secara acak sampai memperoleh satu perubahan valid

Load dan unload merupakan perpindahan melintasi batas keanggotaan arrangement, dengan helper 'load_vehicle()' dan 'unload_vehicle()'. Menukar dua kendaraan yang sama-sama di luar tidak mengubah state karena tidak ada placement yang tersimpan, sehingga dilewati. 'random_neighbor()' mencoba hingga 50 proposal secara default, 'None' berarti belum menemukan perubahan valid dalam batas percobaan, bukan bukti local optimum.

HC menggunakan operasi dasar ini untuk membangkitkan neighborhood, sedangkan SA dan mutasi GA menggunakan 'random_neighbor()'. Cara memilih dan menerima kandidat selanjutnya dibahas pada bagian implementasi masing-masing algoritma.

2.2. Implementasi Algoritma Hill Climbing Steepest Ascent

2.2.1. Gambaran Umum

Hill Climbing merupakan salah satu algoritma pencarian lokal (local search) yang digunakan untuk menyelesaikan permasalahan optimasi dengan mencari solusi terbaik melalui perbaikan secara bertahap. Algoritma ini bekerja dengan memulai pencarian dari suatu kondisi awal (initial state), kemudian mengevaluasi sejumlah solusi tetangga (neighbor) untuk menemukan solusi yang memiliki nilai fungsi objektif lebih baik.

Prinsip kerja Hill Climbing menyerupai proses mendaki bukit, yaitu bergerak dari satu posisi menuju posisi lain yang lebih tinggi hingga tidak ditemukan lagi posisi yang memberikan peningkatan. Dalam konteks optimasi, ketinggian tersebut merepresentasikan nilai fungsi objektif (objective function) yang ingin dimaksimalkan. Pada setiap iterasi, algoritma membandingkan nilai

objektif dari state saat ini dengan neighbor yang tersedia, kemudian menentukan perpindahan berdasarkan aturan varian Hill Climbing yang digunakan.

2.2.2. Input dan Parameter

Input utama algoritma terdiri atas konfigurasi permasalahan (ConfigModel) dan susunan awal kendaraan (initial_arrangement). Konfigurasi mencakup dimensi kapal, kapasitas berat maksimum, daftar kendaraan, serta mode fungsi objektif. Parameter pencarian meliputi:

Parameter	Deskripsi
max_iterations	Batas total iterasi pencarian
on_progress	Callback untuk mengirim perkembangan pencarian
should_cancel	Callback untuk memeriksa permintaan pembatalan

2.2.3. Initial State

Pembentukan initial state dilakukan oleh fungsi initialize_arrangement(). Fungsi tersebut menggunakan generator acak berdasarkan random_seed. Secara default, urutan kendaraan diacak, kemudian setiap kendaraan dicoba ditempatkan menggunakan orientasi dan koordinat acak.

Penempatan diterima apabila:

- Kendaraan berada di dalam geladak.
- Kendaraan tidak bertumpang tindih.
- Kapasitas berat maksimum tidak terlampaui.

Setiap kendaraan memiliki batas percobaan penempatan, secara default sebanyak 100 percobaan. Kendaraan yang tidak berhasil ditempatkan tetap berada di luar kapal. Oleh karena itu, initial state merupakan solusi valid, tetapi belum tentu memuat semua kendaraan yang sebenarnya masih dapat dimasukkan. Program juga menyediakan mode **preserve_positions** untuk mempertahankan koordinat awal yang diberikan pengguna. Namun, eksperimen wajib menggunakan inisialisasi acak sesuai spesifikasi tugas.

2.2.4. Objective Function

Hill Climbing menggunakan calculate_objective() sesuai mode pada konfigurasi sebagaimana dijelaskan pada Subbab 2.1.5. Pada mode shipping fee, fungsi objektifnya adalah:

$$f(s) = \sum_{v \in L(s)} ShippingFee(v)$$

Dengan**:**

- $f(s)$: nilai objektif state s.
- $L(s)$: himpunan kendaraan yang berada di kapal pada state s.
- $ShippingFee(v)$: tarif pengiriman kendaraan v.

Algoritma bertujuan memaksimalkan nilai tersebut. Pada mode bonus ETA priority, penilaian menggunakan rumus prioritas ETA pada Subbab 2.1.5. Aturan pemilihan neighbor tetap mengikuti varian Hill Climbing yang digunakan.

2.2.5. Neighbor Generation

Neighbor dibangkitkan melalui fungsi iter_neighbors(). State direpresentasikan sebagai daftar penempatan kendaraan yang memuat ID, koordinat, panjang, dan lebar. Kendaraan yang tidak tercantum dalam daftar dianggap berada di luar kapal.

Operasi pembangkitan neighbor mencakup:

1. Pemindahan kendaraan: mencoba seluruh koordinat geladak yang memungkinkan bagi kendaraan yang sudah dimuat.
2. Perubahan orientasi: menukar panjang dan lebar kendaraan.
3. Pemindahan keluar kapal: menghapus kendaraan dari daftar penempatan.
4. Pertukaran kendaraan: menukar koordinat dua kendaraan di dalam kapal atau mengganti kendaraan di dalam dengan kendaraan di luar secara atomik.
5. Pemasukan kendaraan: mencoba kendaraan di luar kapal pada seluruh koordinat dan orientasi yang memungkinkan.

Pertukaran dua kendaraan yang sama-sama berada di luar dilewati karena tidak mengubah state dalam representasi yang digunakan. Hanya neighbor

yang valid dan menghasilkan perubahan state yang diteruskan untuk evaluasi. Pembangkitan dilakukan secara bertahap menggunakan iterator sehingga seluruh neighbor tidak harus disimpan sekaligus.

2.2.6. Aturan Pemilihan Neighbor

Keputusan perpindahan bersifat deterministik berdasarkan nilai objektif.

Pada setiap iterasi, algoritma memilih:

$$s^{*} = \arg \max_{s' \in N(s)} f(s')$$

Perpindahan dilakukan hanya jika:

$$f(s^{*}) > f(s)$$

Neighbor yang bernilai sama atau lebih rendah tidak diterima. Jika terdapat beberapa neighbor dengan nilai maksimum yang sama, implementasi mempertahankan neighbor pertama yang ditemukan dalam urutan enumerasi.

Seluruh *neighborhood* tetap diperiksa sebelum perpindahan dilakukan, sehingga algoritma tidak langsung menerima peningkatan pertama.

Pemilihan kandidat terbaik dilakukan dengan membandingkan objective setiap neighbor terhadap nilai tertinggi sementara. Kondisi perbandingan yang ketat membuat kandidat pertama tetap dipertahankan apabila terdapat skor yang sama.

```
highest_obj = current_obj
    highest_neighbor: list[VehiclePlacement] | None = None

    # Di dalam loop evaluasi neighbor:
    if score > highest_obj:
        highest_obj, highest_neighbor = score, neighbor
```

2.2.7. Best State and Stagnation

Selama evaluasi neighborhood, algoritma menyimpan kandidat terbaik melalui variabel *highest_neighbor* dan nilainya melalui *highest_obj*. Kandidat tersebut baru menjadi state saat ini setelah seluruh neighbor selesai dievaluasi.

Pada varian Steepest Ascent, selected dan selected_score merujuk pada highest_neighbor dan highest_obj. Perpindahan dilakukan setelah evaluasi neighborhood selesai:

```
if selected is not None:
        current, current_obj = selected, selected_score
        if current_obj > best_objective:
            best_arrangement, best_objective = current, current_obj
```

Karena perpindahan hanya menerima peningkatan, nilai objektif state yang diterima selalu meningkat. Oleh karena itu, state saat pencarian berhenti juga merupakan state terbaik yang telah dikunjungi. Stagnasi terjadi ketika tidak ditemukan neighbor dengan nilai objektif lebih tinggi. Kondisi ini dapat menunjukkan optimum lokal atau plateau, bukan jaminan tercapainya optimum global. Algoritma tidak menggunakan sideways move maupun random restart untuk keluar dari kondisi tersebut.

2.2.8. Kriteria Penghentian

Pencarian dihentikan ketika:

1. Tidak ditemukan neighbor yang meningkatkan nilai objektif.
2. Jumlah iterasi mencapai max_iterations.

Satu iterasi merupakan satu pemindaian lengkap terhadap neighborhood, termasuk pemindaian terakhir yang tidak menghasilkan peningkatan. Apabila state awal sudah tidak memiliki neighbor yang lebih baik, algoritma tetap mencatat satu iterasi.

2.2.9. History dan Visualisasi

Riwayat pencarian disimpan dalam objective_history. Data pertama mencatat nilai initial state pada iterasi ke-0. Setelah setiap iterasi, nilai objektif state saat ini ditambahkan ke riwayat.

```
history.append(ObjectivePoint(iteration=iterations, objective=current_obj))
    report("iteration", current, current_obj, iterations)
```

Grafik objektif terhadap iterasi bersifat tidak menurun. Titik terakhir dapat memiliki nilai yang sama dengan titik sebelumnya apabila pencarian berhenti setelah pemindaian tanpa peningkatan.

Antarmuka menampilkan:

- Susunan kendaraan pada initial state dan final state.
- Kendaraan yang berada di dalam maupun di luar kapal.
- Nilai objektif dan grafik perkembangannya.
- Jumlah iterasi dan durasi pencarian.

2.2.10. Output

Algoritma mengembalikan objek RunResult yang memuat:

- initial_arrangement dan final_arrangement.
- Daftar kendaraan yang tidak dimuat.
- Nilai objektif awal dan akhir.
- Riwayat nilai objektif.
- Jumlah iterasi.
- Jumlah neighbor yang dievaluasi.
- Alasan penghentian.
- Durasi pencarian dalam milidetik.

Durasi pencarian tidak mencakup pembentukan initial state, akses database, maupun waktu callback visualisasi. Penyimpanan hasil yang selesai dilakukan oleh lapisan service ke MongoDB. Jika opsi Save results to output/ diaktifkan, hasil juga diekspor ke *output/hill_climbing/<run-id>/* berupa JSON dan visualisasi SVG.

2.3. Implementasi Algoritma Genetic Algorithm

2.3.1. Gambaran Umum

Genetic Algorithm (GA) melakukan pencarian melalui populasi. Setiap individu merupakan satu arrangement dengan representasi dan aturan validitas pada Subbab 2.1. Fitness individu menggunakan objective bersama sesuai mode konfigurasi, yaitu total shipping fee pada mode wajib atau total poin prioritas ETA pada mode bonus.

Khusus untuk crossover GA, urutan gen mengikuti seluruh kendaraan pada 'config.vehicles', termasuk yang berada di luar kapal. Dengan demikian, posisi gen selalu merujuk kendaraan yang sama meskipun kedua induk memuat himpunan kendaraan berbeda.

Alur implementasinya adalah membentuk populasi awal, menghitung fitness, mempertahankan satu elite, lalu mengisi sisa populasi melalui roulette, one-point crossover, validasi dengan reject + retry, dan mutasi. Proses diulang hingga jumlah generasi yang ditentukan tercapai.

2.3.2. Input, Parameter, dan Populasi Awal

Fungsi utama 'run_genetic_algorithm()' menerima konfigurasi, settings, arrangement awal, dan RNG. Parameter pencarian yang digunakan adalah sebagai berikut.

Tabel 2.3.1 Daftar Parameter Genetic Algorithm

Parameter	Fungsi
iterations	Jumlah pembentukan generasi baru
population_size	Banyak individu per generasi, sudah termasuk elite
crossover_rate	Peluang crossover untuk setiap pasangan induk
mutation_rate	Peluang mencoba mutasi untuk setiap anak
random_seed	Menentukan rangkaian angka acak agar pencarian dapat direproduksi

Ukuran populasi harus lebih dari satu, jumlah generasi positif, dan kedua rate berada dalam rentang 0 sampai 1. Jumlah elite ditetapkan satu, sedangkan maksimal batas percobaan crossover adalah 20.

```
population = [copy_individual(initial_arrangement)]
for _ in range(settings.population_size - 1):
    check_cancelled(should_stop)

population.append(initialize_arrangement(config,
rng))
```

Arrangement awal dari initializer bersama (Subbab 2.1.4) disalin menjadi individu pertama. Anggota sisanya dibentuk dengan 'initialize_arrangement()' menggunakan RNG yang sama sampai ukuran populasi terpenuhi. Populasi tidak diwajibkan memiliki individu yang semuanya unik.

Populasi ini disebut generasi 0. Individu terbaiknya disimpan sebagai state awal hasil GA, sehingga state awal yang ditampilkan tidak selalu merupakan individu pertama. Mode inisialisasi mengikuti konfigurasi bersama dan untuk eksperimen random, custom starting positions dinonaktifkan.

2.3.3. Evaluasi Fitness

Fitness individu dihitung menggunakan 'calculate_objective()' yang telah dijelaskan pada Subbab 2.1.5. Dalam GA, hasil perhitungan ini dipakai untuk menentukan individu terbaik dan peluang pemilihan induk.

```
fitness = []
for individual in population:
    check_cancelled(should_stop)
    fitness.append(calculate_objective(config,
individual))

# cari fitness tertinggi
# jika seri, pilih acak dari kandidat terbaik
maximum = max(fitness)
best_candidates = []
for i, individual in enumerate(population):
    if fitness[i] == maximum:
        best_candidates.append(individual)
best = rng.choice(best_candidates)
```

Fitness seluruh individu dihitung pada awal evaluasi generasi. Program mengambil nilai maksimum dan memilih satu individu terbaik. Jika beberapa individu memiliki fitness tertinggi yang sama, salah satunya dipilih secara acak.

2.3.4. Roulette Wheel Selection

```
total_fitness = sum(fitness)

if total_fitness == 0:
    # misal semua fitness di populasi nol, setiap
individu dibuat punya peluang yang sama
    index = rng.choice(range(len(population)))
```

```
    draw = index
    kind = "uniform"
else:
    # cth fitness [100, 300, 600] menghasilkan
batas [100, 400, 1000]
    cumulative = []
    running_total = 0
    for score in fitness:
        running_total += score
        cumulative.append(running_total)

    # simpen draw antara 0..1 untuk visualisasi
roulette
    draw = rng.random()
    kind = "weighted"
    target = draw * total_fitness

    # individu terakhir menjadi cadangan jika
pembulatan mencapai batas akhir
    index = len(population) - 1
    for i, upper_bound in enumerate(cumulative):
        if target < upper_bound:
            index = i
            break
```

Seleksi induk menggunakan *Roulette Wheel Selection*, yaitu semakin tinggi fitness, semakin besar bagian individu pada roulette. Jika total fitness positif, peluang individu ke-i adalah

$$P(i) = f(i) / \Sigma f(j)$$

Implementasinya membentuk batas kumulatif, mengambil titik acak pada rentang total fitness, lalu mencari interval yang memuat titik tersebut. Sebagai contoh, fitness '[100, 300, 600]' menghasilkan batas '[100, 400, 1000]'. Jika 'draw = 0.25', titik acaknya 250 sehingga individu kedua terpilih. Jika seluruh fitness nol, pemilihan dilakukan dengan peluang sama.

Roulette dipanggil dua kali untuk memperoleh satu pasangan. Pemilihan dilakukan dengan pengembalian, sehingga pasangan A × B boleh terpilih berulang dan A × A juga diperbolehkan. Namun, crossover A × A menghasilkan susunan yang sama dengan A, tetapi variasi masih dapat muncul melalui mutasi.

2.3.5. One-Point Crossover dan Reject + Retry

Setelah kedua induk terpilih, program mengundi satu angka acak dan membandingkannya dengan 'crossover_rate'. Undian dilakukan sekali per pasangan, bukan per gen. Jika crossover tidak terpilih, kedua anak menyalin induknya masing-masing dan tetap mengikuti tahap mutasi.

```
if draw < settings.crossover_rate:
    children = crossover(config, first, second,
rng, recorder=recorder)
else:
    children = copy_individual(first),
copy_individual(second)
```

Jika crossover dilakukan, placement induk dipetakan berdasarkan ID kendaraan. Titik potong dipilih dengan 'rng.randrange(1, len(config.vehicles))', yaitu dari 1 sampai jumlah kendaraan dikurangi satu. Jika ada lima kendaraan dan titik potongnya dua, pembagiannya menjadi

V1 V2 | V3 V4 V5

Untuk kendaraan kurang dari dua, crossover dilewati.

```
candidate: Individual = []
for vehicle_index, vehicle in
enumerate(config.vehicles):
    # anak 1: awal induk 1, akhir induk 2
    # anak 2: awal induk 2, akhir induk 1
    if vehicle_index < cut:
        source_parent = parents[child_index]
    else:
        # 1 - child_index menukar indeks 0 menjadi
1, dan 1 menjadi 0
        source_parent = parents[1 - child_index]

    placement = source_parent.get(vehicle.id)
    if placement is not None:

candidate.append(placement.model_copy(deep=True))
```

Anak pertama mengambil bagian awal dari induk pertama dan bagian akhir dari induk kedua. Anak kedua mengambil sumber yang berlawanan. Ekspresi '1 - child_index' menukar indeks induk 0 dan 1. Jika placement tidak ditemukan pada induk sumber, kendaraan tersebut tetap berada di luar kapal. Placement yang diwariskan disalin agar tidak mengubah induk.

Penggabungan placement dapat menghasilkan overlap atau pelanggaran berat meskipun kedua induknya valid. Karena itu, setiap kandidat diperiksa melalui 'validate_arrangement()'. Anak valid disimpan, sedangkan anak tidak valid ditolak dan dicoba lagi dengan titik potong baru. Induk tetap sama dan peluang crossover tidak diundi ulang. Anak yang sudah valid tidak dibuat ulang, kedua anak yang diterima dapat berasal dari percobaan berbeda.

```
first_child, second_child = children
if first_child is None:
    first_child = copy_individual(first)
if second_child is None:
    second_child = copy_individual(second)
return first_child, second_child
```

Retry berhenti ketika kedua anak valid atau sudah mencapai 20 percobaan. Hanya anak yang masih belum valid yang diganti salinan induk terkait. Jadi, jika anak pertama gagal tetapi anak kedua valid, anak pertama menyalin induk pertama dan anak kedua tetap dipakai. Batas ini memastikan reproduksi selesai, meskipun fallback yang terlalu sering dapat mengurangi variasi populasi.

2.3.6. Mutasi dengan Random Neighbor

```
draw = rng.random()
result = child
operation = "not selected"
changed_ids = []

if draw < rate:
    changed = random_neighbor(config, child, rng)
    if changed is not None:
        result = changed
        if recorder is not None and
recorder.enabled:
            operation, changed_ids =
mutation_change(child, changed)
    else:
        operation = "no valid neighbor"
```

Fungsi 'mutate()' mengundi mutasi secara terpisah untuk setiap anak setelah crossover atau penyalinan induk. Jika angka acaknya lebih kecil dari 'mutation_rate', program memanggil 'random_neighbor()'. Neighbor valid

menjadi susunan anak yang baru, jika tidak ditemukan, anak tetap seperti sebelumnya.

Jenis perubahan mengikuti 'random_neighbor()' pada Subbab 2.1.6. GA memakai hasil validnya sebagai mutasi anak, tanpa menambah aturan bahwa perubahan tersebut harus meningkatkan fitness.

Mutasi tidak harus meningkatkan fitness. Anak dengan fitness lebih rendah tetap diperbolehkan, sementara perpindahan atau rotasi dapat mengubah posisi tanpa mengubah fitness. Kedua rate merupakan peluang, bukan jatah pasti: mutation rate 1.0 berarti setiap anak mencoba mutasi, tetapi belum tentu berhasil berubah.

2.3.7. Elitism dan Pergantian Generasi

```
next_population = [copy_individual(best)]
```

Satu individu terbaik langsung disalin ke populasi berikutnya tanpa crossover dan mutasi. Elite dipilih ulang setiap generasi, bukan hanya pada generasi awal. Jika muncul anak yang lebih baik, anak tersebut dapat menjadi elite berikutnya. Individu sumber elite juga tetap boleh dipilih sebagai induk melalui roulette.

```
offspring = []
for i, child in enumerate(children):
    mutated_child = mutate(
        config, child, settings.mutation_rate, rng,
recorder, i + 1
    )
    offspring.append(mutated_child)

remaining_slots = settings.population_size -
len(next_population)
if remaining_slots == 1:
    # Kedua anak sudah diproses. Pilih acak satu
untuk slot terakhir.
    chosen = rng.choice(offspring)
    admitted = []
    for i, child in enumerate(offspring):
        if child is chosen:
            admitted.append(i + 1)
            break
    next_population.append(chosen)
else:
```

```
    admitted = [1, 2]
    next_population.extend(offspring)
```

Setelah satu slot ditempati elite, slot lainnya diisi anak hasil reproduksi. Kedua anak masuk jika slot mencukupi. Jika tinggal satu slot, salah satu anak dipilih secara acak, tanpa membandingkan fitness keduanya. Tidak ada seleksi tambahan berdasarkan fitness untuk menerima anak ke populasi baru.

Sebagai contoh, populasi berukuran lima berisi satu elite dan empat anak dari dua pasangan. Elite sudah termasuk ukuran populasi, sehingga jumlah individu tetap lima. Setelah penuh, 'population = next_population' mengganti populasi lama.

Elitism menjaga maksimum fitness agar tidak turun karena solusi terbaik tetap terbawa. Rata-rata fitness masih dapat turun karena anak yang lebih buruk diperbolehkan masuk. Mekanisme ini menjaga solusi terbaik yang ditemukan, tetapi tidak menjamin optimum global.

2.3.8. Kondisi Berhenti, History, dan Output

```
for generation in range(settings.iterations + 1):
    check_cancelled(should_stop)
    recorder.generation = generation
    recorder.pair = None
```

Loop mengevaluasi generasi 0 sampai 'settings.iterations'. Input 100 berarti 100 kali pembentukan generasi baru dan 101 titik statistik. Setelah generasi terakhir dievaluasi, proses berhenti tanpa reproduksi tambahan. Tidak ada penghentian otomatis berdasarkan stagnasi, suhu, atau target fitness.

Setiap generasi mencatat maksimum dan rata-rata fitness. 'objective_history' berisi maksimum, sedangkan 'population_history' menyimpan keduanya. Hasil 'RunResult' juga memuat arrangement awal terbaik generasi 0, arrangement akhir terbaik generasi terakhir, daftar kendaraan di luar kapal, jumlah generasi selesai, dan durasi pencarian. Karena elitism, hasil terbaik yang pernah ditemukan tetap terbawa sampai akhir.

Durasi 'runtime_ms' memasukkan pembentukan anggota populasi tambahan, evaluasi, reproduksi, dan callback progress. Pembuatan arrangement pertama sebelum mesin GA dipanggil serta penyimpanan database tidak termasuk. Perbedaan cakupan waktu ini perlu dijelaskan ketika membandingkan hasil dengan algoritma lain.

Tombol Stop ditangani melalui callback 'should_stop' pada checkpoint pencarian. Pembatalan bersifat kooperatif dan hasil parsial tidak disimpan sebagai run selesai. Hasil normal disimpan otomatis ke MongoDB setelah pencarian selesai.

2.3.9. Visualisasi dan Pengujian

Frontend menampilkan progress, arrangement awal/akhir pada viewer 3D dan grid 2D, grafik maksimum/rata-rata fitness, serta riwayat run. Mode demo merekam keputusan aktual melalui 'DemoRecorder', sehingga roulette, titik potong, alasan penolakan anak, mutasi, dan pergantian populasi dapat ditinjau kembali. Replay membaca rekaman tanpa menambah undian random. Perekaman demo dibatasi hingga 10 individu, 10 generasi, dan 30 kendaraan, serta dinonaktifkan untuk eksperimen kinerja.

Pengujian implementasi mencakup validitas solusi, hasil yang dapat direproduksi, ukuran populasi, maksimum fitness yang tidak menurun, roulette, retry dan fallback, mutasi, pembatalan, serta kesesuaian trace demo. Pengujian tersebut tersedia pada tes backend GA/job dan tes frontend replay. Kelulusan tes memeriksa perilaku kode, bukan membuktikan solusi optimum global.

Eksperimen parameter dan analisis hasil dibahas pada bagian eksperimen tersendiri agar tidak mengulang penjelasan implementasi. Hasil numeriknya belum dicantumkan karena eksperimen belum dilaksanakan.

2.4. Implementasi Algoritma Simulated Annealing

2.4.1. Gambaran Umum

Simulated Annealing mencari konfigurasi pemuatan kendaraan yang memaksimalkan objective sesuai mode konfigurasi, yaitu total tarif pengiriman atau total poin prioritas ETA. Algoritma mempertahankan satu solusi saat ini (current state), kemudian membangkitkan solusi tetangga melalui perubahan posisi, pertukaran kendaraan, atau perubahan orientasi.

Solusi tetangga yang memiliki nilai objektif lebih tinggi atau sama selalu diterima. Solusi yang lebih buruk masih dapat diterima berdasarkan probabilitas yang dipengaruhi temperatur. Mekanisme ini memungkinkan pencarian keluar dari konfigurasi yang sulit diperbaiki melalui perpindahan langsung.

Pada temperatur tinggi, peluang menerima penurunan nilai objektif lebih besar. Temperatur diturunkan secara bertahap sehingga pencarian semakin selektif. Implementasi juga menyimpan solusi terbaik sepanjang pencarian agar hasil terbaik tidak hilang ketika algoritma menerima solusi yang lebih buruk.

2.4.2. Input dan Parameter

Fungsi utama menerima konfigurasi kapal dan kendaraan, parameter algoritma, serta pembangkit bilangan acak. Fungsi ini juga dapat menerima susunan awal dan fungsi pemantau untuk mengirim perkembangan pencarian.

```
def simulated_annealing(
    config: ConfigModel,
    settings: SimulatedAnnealingSettings,
    rng: Random,
    initial_arrangement: list[VehiclePlacement] |
None = None,
    on_snapshot: Callable[[StateSnapshot], None] |
None = None,
) -> RunResult:
```

Parameter algoritma didefinisikan melalui model berikut:

```
class SimulatedAnnealingSettings(BaseModel):
    algorithm: Literal["simulated_annealing"]
```

```
    iterations: int = Field(gt=0, le=10000)
    initial_temperature: float = Field(
        gt=0, le=1e12, allow_inf_nan=False
    )
    cooling_rate: float = Field(
        gt=0, lt=1, allow_inf_nan=False
    )
    minimum_temperature: float = Field(
        default=0.001, ge=1e-12,
allow_inf_nan=False
    )
    stagnation_window: int = Field(default=100,
gt=0)
```

Parameter	Fungsi
iterations	Batas maksimum iterasi pencarian, paling banyak 10.000.
initial_temperature	Temperatur awal yang mengendalikan penerimaan solusi lebih buruk.
cooling_rate	Faktor pengali temperatur pada akhir setiap iterasi, dengan nilai antara 0 dan 1.
minimum_temperature	Ambang temperatur untuk menghentikan pencarian dengan nilai bawaannya 0,001.
stagnation_window	Jumlah iterasi tanpa perbaikan nilai terbaik untuk mencatat satu episode stagnasi dengan nilai bawaan 100.

2.4.3. Initial State

Simulated Annealing menggunakan initialize_arrangement() yang telah dijelaskan pada Subbab 2.1. Jika initial_arrangement diberikan, algoritma menggunakan salinan susunan tersebut. Initial state kemudian divalidasi dan dihitung nilai objective-nya.

```
current = (
    initialize_arrangement(config, rng)
    if initial_arrangement is None
    else [p.model_copy(deep=True) for p in
initial_arrangement]
)

errors = validate_arrangement(config, current)
```

```
if errors:
    raise ValueError("; ".join(errors))

initial = [p.model_copy(deep=True) for p in
current]
score = initial_score = calculate_objective(config,
current)
best, best_score = current, score
temperature = settings.initial_temperature
```

Variabel current dan score menyimpan solusi saat ini, sedangkan best dan best_score menyimpan solusi terbaik sepanjang pencarian. Keduanya menggunakan solusi yang sama pada awal pencarian. Variabel initial menyimpan salinan susunan awal, sementara temperatur diambil dari parameter initial_temperature.

2.4.4. Objective Function

Simulated Annealing menggunakan fungsi calculate_objective() yang dijelaskan pada Subbab 2.1.5. Fungsi ini menghitung total shipping fee atau total poin prioritas ETA sesuai objective_mode pada konfigurasi. Nilai tersebut digunakan untuk menghitung selisih kualitas kandidat terhadap state saat ini dan memperbarui solusi terbaik. Pergantian mode objective tidak mengubah rumus probabilitas penerimaan maupun jadwal pendinginan.

2.4.5. Neighbor Generation

Pada setiap iterasi, algoritma memanggil random_neighbor() untuk memperoleh satu kandidat valid melalui operasi move, swap, atau rotate. Detail operasi dan validasinya telah dijelaskan pada Subbab 2.1.

```
candidate = random_neighbor(config, current, rng)
```

2.4.6. Acceptance Probability

Berdasarkan selisih objective pada Subbab 2.4.4, kandidat dengan $\Delta E \geq 0$ selalu diterima. Kandidat dengan $\Delta E < 0$ diterima berdasarkan probabilitas yang dipengaruhi temperatur:

$$P(\text{diterima}) = \begin{cases} 1, & \Delta E \geq 0, \\ e^{\Delta E/T}, & \Delta E < 0. \end{cases}$$

Aturan tersebut diimplementasikan melalui fungsi berikut:

```
def acceptance_probability(delta: int, temperature:
float) -> float:
    if not math.isfinite(temperature) or
temperature <= 0:
        raise ValueError("Temperature must be
finite and positive.")
    return 1.0 if delta >= 0 else math.exp(delta /
temperature)
```

Pada temperatur tinggi, kandidat yang lebih buruk memiliki peluang penerimaan lebih besar. Sebagai contoh, penurunan objective sebesar 10 pada temperatur 100 memiliki probabilitas sekitar 0,9048. Pada temperatur 1, probabilitasnya turun menjadi sekitar 0,0000454.

Keputusan penerimaan diterapkan melalui kode berikut:

```
probability = (
    0.0 if delta is None
    else acceptance_probability(delta, temperature)
)

accepted = (
    candidate is not None
    and delta is not None
    and (delta >= 0 or rng.random() < probability)
)

if accepted and candidate is not None and delta is
not None:
    current = candidate
    score += delta
```

Untuk kandidat yang lebih buruk, rng.random() menghasilkan bilangan acak pada rentang $[0, 1)$. Kandidat diterima apabila bilangan tersebut lebih kecil daripada probabilitas penerimaan. Kandidat dengan nilai sama selalu diterima, sedangkan kandidat yang ditolak tidak mengubah current state.

2.4.7. Best State and Stagnation

Algoritma membedakan solusi saat ini dan solusi terbaik. Solusi saat ini dapat mengalami penurunan nilai objektif, sedangkan solusi terbaik hanya diperbarui ketika ditemukan nilai yang lebih tinggi.

```
if score > best_score:
```

```
    best, best_score = current, score
    stagnant, in_stagnation = 0, False
else:
    stagnant += 1
    if stagnant >= settings.stagnation_window and
not in_stagnation:
        stuck_count += 1
        in_stagnation = True
```

Variabel stagnant menghitung iterasi tanpa perbaikan nilai terbaik. Ketika jumlahnya mencapai stagnation_window, program menambah stuck_count satu kali dan menandai bahwa pencarian sedang berada dalam episode stagnasi.

Episode yang sama tidak dihitung berulang kali. Penghitungan diaktifkan kembali setelah ditemukan solusi terbaik yang baru.

Dalam implementasi ini, stuck_count merupakan jumlah episode stagnasi berdasarkan tidak adanya perbaikan nilai terbaik. Metrik tersebut digunakan sebagai indikator operasional kondisi stuck, bukan bukti matematis bahwa solusi berada pada optimum lokal.

2.4.8. Cooling Schedule and Stopping Criteria

Implementasi menggunakan pendinginan geometrik:

$$T_{k+1} = \alpha T_k, 0 < \alpha < 1,$$

dengan T_k merupakan temperatur pada iterasi saat ini dan α merupakan cooling_rate.

```
temperature *= settings.cooling_rate
```

Nilai cooling_rate yang mendekati 1 mempertahankan temperatur tinggi lebih lama. Nilai yang lebih kecil menurunkan temperatur lebih cepat.

Pencarian berhenti apabila jumlah iterasi maksimum tercapai atau temperatur berada di bawah minimum_temperature pada awal iterasi.

```
stop_reason = "iteration_limit"
for iteration in range(1, settings.iterations + 1):
```

```
    if temperature < settings.minimum_temperature:
        stop_reason = "minimum_temperature"
        break
```

Pendinginan tetap dilakukan pada akhir setiap iterasi meskipun kandidat ditolak atau tidak ditemukan. Stagnasi tidak menjadi kondisi penghentian pencarian. Jika temperatur awal sudah lebih rendah daripada temperatur minimum, algoritma mengembalikan solusi awal tanpa menjalankan iterasi pencarian.

2.4.9. History dan Visualisasi

Setiap iterasi menyimpan informasi temperatur, nilai objektif saat ini, nilai terbaik, selisih objektif, probabilitas penerimaan, dan keputusan penerimaan.

```
history.append(
    AnnealingPoint(
        iteration=iteration,
        temperature=temperature,
        objective=score,
        best_objective=best_score,
        delta=delta,
        log_acceptance=log_acceptance,
        exponential=exponential,
        acceptance_probability=probability,
        accepted=accepted,
        candidate_found=candidate is not None,
    )
)
objectives.append(
    ObjectivePoint(iteration=iteration,
objective=score)
)
```

Program juga mencatat nilai $e^{\Delta E/T}$ untuk kebutuhan grafik eksperimen:

```
log_acceptance = None if delta is None else delta /
temperature
exponential = (
    None
    if log_acceptance is None or log_acceptance >
709
    else math.exp(log_acceptance)
)
```

exponential merupakan nilai eksponensial mentah dan dapat lebih besar dari 1 ketika $\Delta E > 0$. Sebaliknya, acceptance_probability selalu berada pada rentang 0 sampai 1.

Untuk menghindari *overflow*, nilai eksponensial disimpan sebagai None apabila eksponennya lebih besar dari 709. Nilai $\Delta E/T$ tetap disimpan melalui log_acceptance. Jika kandidat tidak ditemukan, kedua nilai tersebut juga kosong.

Selain riwayat numerik pada setiap iterasi, algoritma menyimpan *snapshot* susunan secara berkala:

```
snapshot_interval = max(1,
math.ceil(settings.iterations / 100))

if iteration % snapshot_interval == 0:
    snapshots.append(
        StateSnapshot(
            iteration=iteration,
            objective=score,
            arrangement=current,
        )
    )
    notify(snapshots[-1])
```

Penyimpanan berkala membatasi jumlah susunan untuk pemutaran ulang, sementara metrik numerik tetap dicatat pada seluruh iterasi. Snapshot awal dan keadaan terakhir juga disimpan.

Pada antarmuka, riwayat digunakan untuk menampilkan grafik current objective, best objective, dan nilai eksponensial, serta replay susunan kendaraan. Ketika terjadi overflow, grafik eksponensial menggunakan nilai $\Delta E/T$ sebagai pengganti nilai eksponensial mentah.

Grafik current objective dapat naik atau turun karena algoritma dapat menerima solusi yang lebih buruk, sedangkan best objective tidak menurun. Snapshot terakhir menunjukkan last state, yang dapat berbeda dari best state pada hasil akhir.

2.4.10. Output

Algoritma mengembalikan solusi terbaik sepanjang pencarian sebagai hasil akhir. Keadaan terakhir juga disimpan karena nilainya dapat lebih rendah daripada solusi terbaik.

```
loaded = {p.vehicle_id for p in best}
return RunResult(
    initial_arrangement=initial,
    final_arrangement=best,
    last_arrangement=current,
    unloaded_vehicle_ids=[
        v.id for v in config.vehicles if v.id not
in loaded
    ],
    initial_objective=initial_score,
    final_objective=best_score,
    best_objective=best_score,
    objective_history=objectives,
    runtime_ms=(perf_counter() - started -
observer_seconds) * 1000,
    iterations_completed=completed,
    stuck_count=stuck_count,
    stop_reason=stop_reason,
    annealing_history=history,
    snapshots=snapshots,
)
```

final_arrangement dan final_objective menyimpan solusi terbaik sepanjang pencarian, sedangkan last_arrangement menyimpan keadaan ketika pencarian berhenti. Daftar kendaraan yang tidak dimuat ditentukan berdasarkan solusi terbaik. Output juga mencatat jumlah iterasi, episode stagnasi, alasan berhenti, riwayat pencarian, dan durasi.

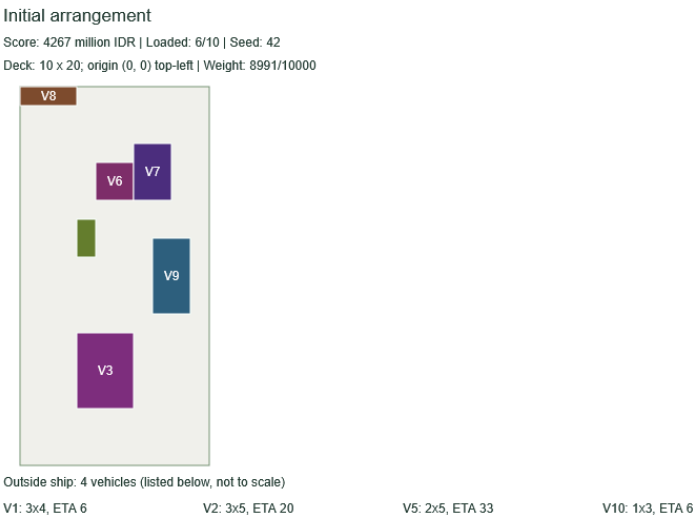
Durasi dihitung menggunakan perf_counter(), mencakup inisialisasi internal dan proses pencarian, dengan mengurangi waktu pelaksanaan callback on_snapshot. Penyimpanan database di luar fungsi tidak termasuk dalam pengukuran.

Karena solusi awal menjadi best state pertama, final objective tidak lebih rendah daripada initial objective. Namun, hasil tersebut belum tentu merupakan global optimum. Analisis hasil pencarian dibahas pada bagian eksperimen.

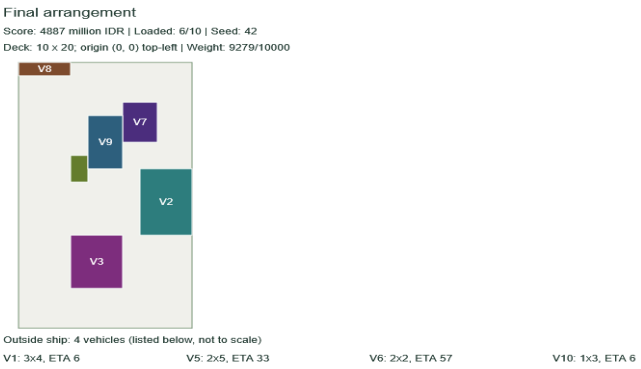
2.5. Eksperimen Algoritma Hill Climbing Steepest Ascent

Eksperimen dijalankan tiga kali menggunakan konfigurasi kapal dan kendaraan yang sama, dengan seed 42, 43, dan 44 untuk menghasilkan susunan awal berbeda. Batas maksimum pencarian ditetapkan sebesar 100 iterasi.

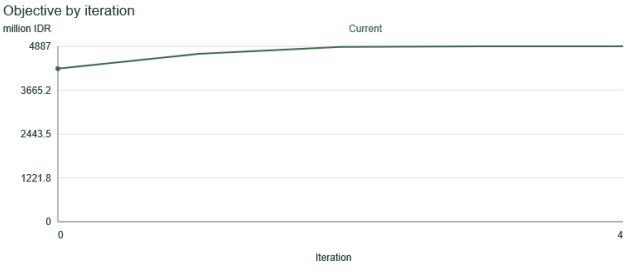
- Eksperimen 1
  - State Awal



  - State Akhir



  - Objective Function



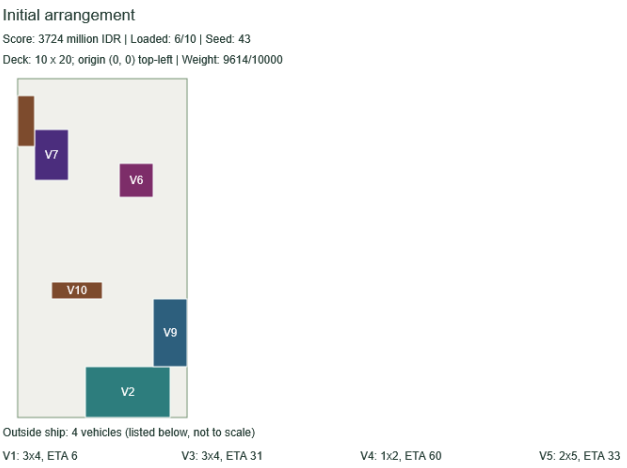
- Durasi Proses Pencarian

  Eksperimen 1 memiliki durasi 530.6227800028864 milisekon.

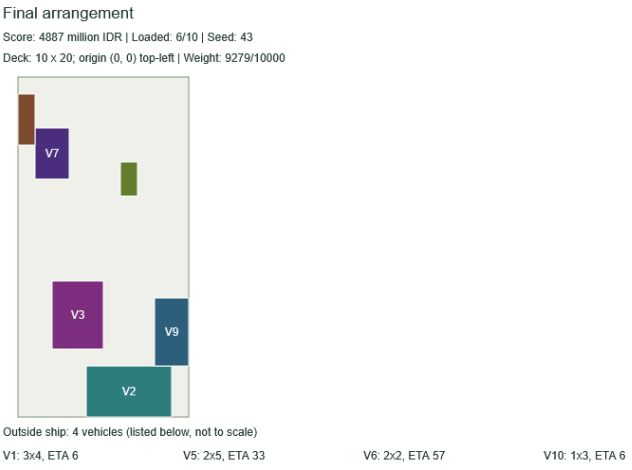
- Iterasi Hingga Berhenti

  Eksperimen 1 mengiterasi sebanyak 4 kali sebelum berhenti

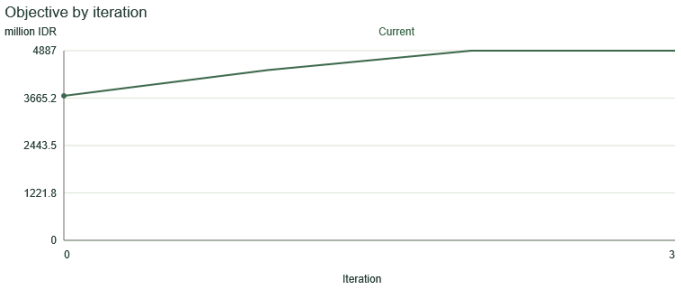
- Eksperimen 2
  - State Awal



  - State Akhir



  - Objective Function



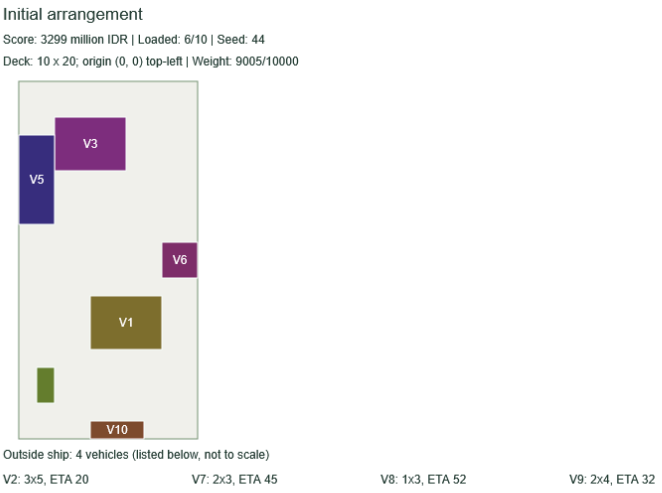
- Durasi Proses Pencarian

  Eksperimen 2 memiliki durasi 367.42772700381465 milisekon.

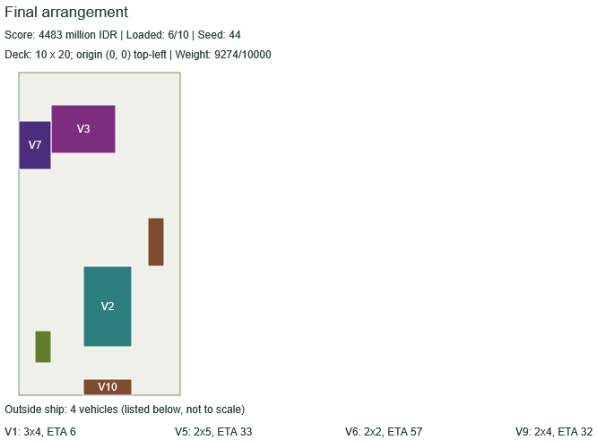
- Iterasi Hingga Berhenti

  Eksperimen 2 mengiterasi sebanyak 3 kali sebelum berhenti

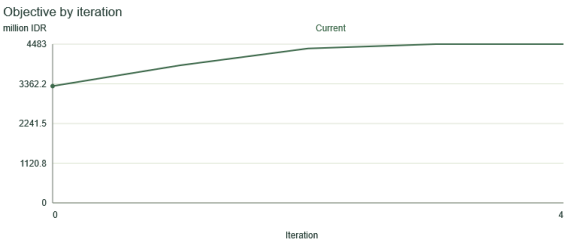
- Eksperimen 3
  - State Awal



  - State Akhir



- Objective Function



- Durasi Proses Pencarian

  Eksperimen 3 memiliki durasi 480.2063690003706 milisekon.

- Iterasi Hingga Berhenti

  Eksperimen 3 mengiterasi sebanyak 4 kali sebelum berhenti

- Kesimpulan Eksperimen

Percobaan	**Seed**	**Objective awal**	**Objective akhir**	**Iterasi hingga berhenti**	**Durasi pencarian**
1	42	4.267	4.887	4	530,62 ms
2	43	3.724	4.887	3	367,43 ms

3	44	3.299	4.483	4	480,21 ms

Ketiga percobaan berhenti karena tidak ditemukan neighbor dengan objective lebih tinggi, bukan karena mencapai batas 100 iterasi. Jumlah iterasi mencakup pemindaian terakhir yang tidak menghasilkan peningkatan.

Hasil eksperimen dapat dilihat di folder output/hill-climbing/:

- 78be0995-5e40-432d-a83c-4a72ca5a649e/ (eksperimen 1)
- 6764e82c-4192-4a5f-8c13-dd5d81d85a3c/ (eksperimen 2)
- 8d0f35d9-ade4-4a40-944a-e49f65ea76ea/ (eksperimen 3)

2.6. Eksperimen Algoritma Genetic Algorithm

2.6.1. Tujuan dan konfigurasi eksperimen

Eksperimen dilakukan untuk mengamati pengaruh jumlah generasi dan ukuran populasi terhadap nilai objective akhir, konsistensi hasil, dan durasi pencarian Genetic Algorithm (GA). Seluruh eksperimen menggunakan konfigurasi yang sama, yaitu geladak berukuran 10 × 20 grid, kapasitas berat 10.000 unit, serta 10 kendaraan pada input/config.json. Fungsi objektif yang digunakan adalah total shipping fee kendaraan yang dimuat, dengan satuan juta rupiah. Objective bonus ETA tidak digunakan pada eksperimen ini.

Pengujian mencakup enam kombinasi parameter, masing-masing dijalankan dengan seed 42, 43, dan 44, sehingga terdapat 18 run unik. Pengaruh jumlah generasi diuji pada populasi 30 dengan batas 50, 100, dan 200 generasi. Pengaruh ukuran populasi diuji pada batas 100 generasi dengan populasi 10, 20, 30, dan 40. Kombinasi populasi 30 dan 100 generasi menjadi titik acuan bersama kedua pengujian dan tidak dihitung dua kali.

Parameter tetap	Nilai
Probabilitas crossover	0,8 per pasangan induk
Probabilitas mutasi	0,3 per anak
Seleksi	Roulette wheel selection

Crossover	One-point; reject + retry hingga 20 percobaan
Elitisme	Satu individu terbaik per generasi
Inisialisasi	Acak; preserve_positions = false
Rekaman demo	Nonaktif

Satu individu merepresentasikan susunan kendaraan yang dimuat; kendaraan yang tidak tercantum dalam placement dianggap berada di luar kapal. Urutan gen untuk crossover mengikuti urutan seluruh kendaraan pada konfigurasi. Anak hasil crossover harus memenuhi batas geladak, larangan overlap, dan batas berat. Anak yang masih tidak valid setelah batas percobaan menyalin induknya. Mutasi menggunakan satu neighbor valid berupa swap, perpindahan (termasuk masuk atau keluar kapal), atau perubahan orientasi. Karena mutasi tidak mensyaratkan perbaikan fitness, rata-rata populasi dapat turun. Satu elite menjaga agar fitness maksimum tidak turun.

Generasi 0 adalah populasi awal. Oleh karena itu, batas G generasi menghasilkan G + 1 titik statistik. State awal yang dilaporkan GA merupakan individu terbaik dalam populasi awal, bukan seluruh populasi dan bukan selalu individu pertama yang dibangkitkan. Ukuran populasi yang berbeda dapat menghasilkan objective awal berbeda meskipun seed sama. Pencarian tetap berjalan sampai batas generasi, meskipun nilai terbaik sudah tidak meningkat.
Durasi berasal dari runtime_ms hasil run, bukan waktu yang dibutuhkan untuk menampilkan animasi atau menyimpan berkas. Pada implementasi GA, timer mencakup validasi input dan pembangkitan sisa populasi di dalam fungsi GA; pembangkitan individu pertama dilakukan sebelumnya oleh job service. Perbedaan cakupan waktu ini perlu dipertimbangkan jika membandingkan durasi antaralgoritma.

2.6.2. Rekapitulasi seluruh percobaan

Pada tabel berikut, g* menunjukkan generasi pertama ketika fitness maksimum mencapai 4.887. Tanda "—" berarti nilai tersebut belum tercapai sampai batas generasi. Durasi dibulatkan menjadi dua angka desimal.

Populasi	Generasi	Seed	Objective awal	Objective akhir	Durasi (ms)	g*
30	50	42	4.501	4.887	518,17	9
30	50	43	4.763	4.887	485,98	24
30	50	44	4.867	4.887	433,81	8
30	100	42	4.501	4.887	1.000,37	9
30	100	43	4.763	4.887	889,37	24
30	100	44	4.867	4.887	917,33	8
30	200	42	4.501	4.887	1.756,40	9
30	200	43	4.763	4.887	1.839,91	24
30	200	44	4.867	4.887	1.686,58	8
10	100	42	4.501	4.887	304,30	42
10	100	43	4.427	4.887	280,18	60
10	100	44	4.392	4.649	305,18	—
20	100	42	4.501	4.887	610,69	8
20	100	43	4.763	4.887	543,11	5
20	100	44	4.583	4.887	558,84	69
40	100	42	4.501	4.887	1.231,34	50
40	100	43	4.763	4.887	1.317,35	25
40	100	44	4.867	4.887	1.264,70	25

2.6.3. Pengaruh jumlah generasi

Populasi	Generasi	Rata-rata objective akhir	Simpangan baku sampel	Rata-rata durasi (ms)	Mencapai 4.887
30	50	4.887,00	0,00	479,32	3/3
30	100	4.887,00	0,00	935,69	3/3
30	200	4.887,00	0,00	1.760,96	3/3

Pada populasi 30, seluruh seed mencapai objective akhir 4.887 untuk ketiga batas generasi. Penambahan batas dari 50 menjadi 100 atau 200 generasi tidak

meningkatkan objective akhir pada dataset ini. Riwayat fitness pada seed yang sama memiliki awalan identik untuk ketiga batas generasi; penambahan batas hanya memperpanjang pencarian setelah nilai terbaik tercapai. Waktu rata-rata meningkat dari 479,32 ms menjadi 935,69 ms dan 1.760,96 ms. Dengan demikian, batas 50 generasi sudah mencukupi untuk tiga seed yang diuji pada populasi 30.

Elitisme menjelaskan mengapa garis fitness maksimum tetap datar setelah nilai terbaik ditemukan. Garis rata-rata dapat berfluktuasi karena anak tetap diterima meskipun fitness-nya lebih rendah dan mutasi dapat menurunkan fitness. Kurva maksimum yang datar tidak berarti seluruh individu identik atau seluruh populasi memiliki fitness optimal.

Pada populasi 30, optimum pertama kali tercapai pada generasi 9, 24, dan 8 untuk seed 42, 43, dan 44. Dengan demikian, bahkan batas 50 generasi masih menyisakan generasi setelah optimum tercapai. Eksperimen tidak menguji penghentian dini; durasi pada tabel tetap durasi seluruh batas generasi.

2.6.4. Pengaruh ukuran populasi

Populasi	Generasi	Rata-rata objective akhir	Simpangan baku sampel	Rata-rata durasi (ms)	Mencapai 4.887
10	100	4.807,67	137,41	296,55	2/3
20	100	4.887,00	0,00	570,88	3/3
30	100	4.887,00	0,00	935,69	3/3
40	100	4.887,00	0,00	1.271,13	3/3

Pada batas 100 generasi, populasi 10 menghasilkan objective 4.887, 4.887, dan 4.649. Rata-ratanya 4.807,67 dengan simpangan baku sampel 137,41. Populasi 20, 30, dan 40 mencapai 4.887 pada seluruh seed, sehingga simpangan baku objective akhir masing-masing nol. Hal ini menunjukkan bahwa populasi lebih besar memberikan hasil akhir yang lebih konsisten pada tiga pengulangan ini. Namun, jumlah pengulangan tersebut belum cukup untuk menyimpulkan probabilitas keberhasilan secara umum.

Populasi 10 memiliki waktu rata-rata paling singkat, yaitu 296,55 ms. Populasi 20 membutuhkan 570,88 ms, populasi 30 membutuhkan 935,69 ms, dan populasi 40 membutuhkan 1.271,13 ms. Bertambahnya populasi meningkatkan jumlah individu yang dievaluasi dan anak yang diproses setiap generasi. Di antara ukuran populasi yang diuji pada 100 generasi, populasi 20 memberikan hasil akhir sama dengan populasi 30 dan 40 dengan durasi lebih singkat. Di antara seluruh enam kombinasi, populasi 30 dengan 50 generasi memiliki rata-rata durasi terendah yang mencapai 4.887 pada ketiga seed.

Run populasi 10 dan seed 44 mencapai nilai akhir 4.649, yaitu 238 atau sekitar 4,87% di bawah 4.887. Kebuntuan ini dapat berkaitan dengan keterbatasan eksplorasi populasi dan perubahan keanggotaan kendaraan akibat operator genetika, tetapi penyebab spesifiknya tidak dapat dipastikan dari statistik maksimum/rata-rata saja karena rekaman detail reproduksi dinonaktifkan. Nilai akhir yang konstan tidak membuktikan local optimum atau hilangnya seluruh keragaman populasi.

Populasi lebih besar juga tidak selalu mencapai optimum pada generasi lebih awal. Populasi 40 mencapai optimum pada generasi 50, 25, dan 25, sedangkan populasi 30 pada generasi 9, 24, dan 8. Run populasi 10 dan seed 44 sudah mencapai nilai 4.649 pada generasi 38 dan tidak membaik hingga generasi 100. Angka ini menunjukkan bahwa hasil akhir dan kecepatan konvergensi perlu dibaca terpisah; seed yang sama pada ukuran populasi berbeda juga tidak menyamakan seluruh urutan keputusan acak.

2.6.5. Kualitas solusi dan batas optimum

Untuk konfigurasi ini, nilai 4.887 dapat dibuktikan sebagai optimum global. Pemeriksaan seluruh 2^10 = 1.024 subset kendaraan, dengan hanya mempertimbangkan batas berat 10.000, menghasilkan batas atas shipping fee sebesar 4.887. Mengabaikan batas spasial memperluas ruang solusi, sehingga tidak ada susunan valid yang dapat melampaui batas atas tersebut. Karena hasil run memuat susunan spasial valid dengan shipping fee 4.887, batas atas itu tercapai dan menjadi optimum global untuk konfigurasi eksperimen ini.

Sebanyak 17 dari 18 run mencapai optimum tersebut. Angka ini merupakan ringkasan enam kombinasi parameter pada satu konfigurasi, bukan estimasi keberhasilan untuk semua persoalan. Seluruh run optimal memuat enam kendaraan; validasi state awal dan akhir memastikan tidak ada overlap, kendaraan keluar batas, atau pelanggaran kapasitas. Simpangan baku objective akhir nol pada suatu kombinasi hanya menunjukkan kesamaan nilai akhir, bukan kesamaan koordinat atau perjalanan pencarian.

Dataset hanya menggunakan 10 kendaraan, dengan total luas seluruh kendaraan 85 grid dari luas geladak 200 grid dan total berat 15.322 unit. Karena seluruh kendaraan tidak dapat dimuat sekaligus berdasarkan berat, pemilihan kendaraan berperan penting. Luas total yang kecil tidak dengan sendirinya menjamin sembarang susunan bebas overlap. Hasil pada konfigurasi ini belum mewakili kasus geladak padat atau jumlah kendaraan besar.

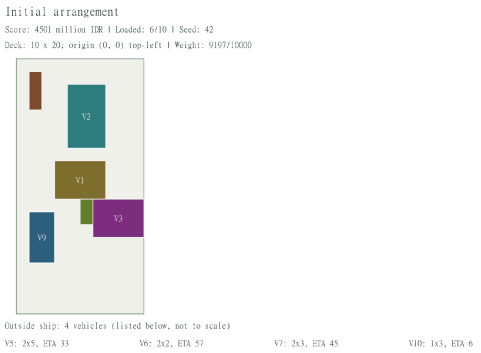
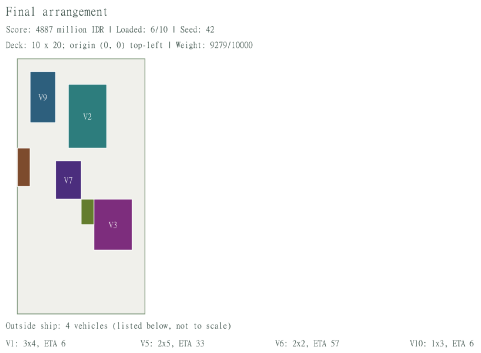
2.6.6. Visualisasi state dan perkembangan objective

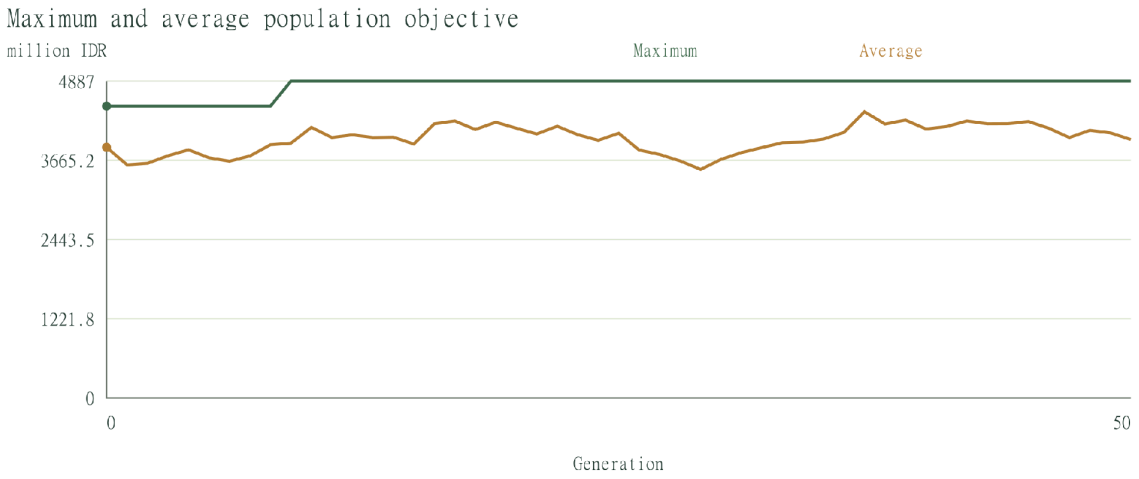
Visualisasi berikut menggunakan berkas yang diekspor program dari run asli. Setiap percobaan menampilkan state awal terbaik, state akhir terbaik, serta grafik fitness maksimum dan rata-rata populasi terhadap generasi. Identitas run dan lokasi sumber disertakan agar hasil dapat ditelusuri.

Percobaan 1: populasi 30, 50 generasi, seed 42

Objective 4.501 → 4.887; durasi 518,17 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: 6791d55f-a51c-4cae-bd0d-7359572ef9ce. Sumber: output/genetic_algorithm/6791d55f-a51c-4cae-bd0d-7359572ef9ce/run.json.

State awal	State akhir
	

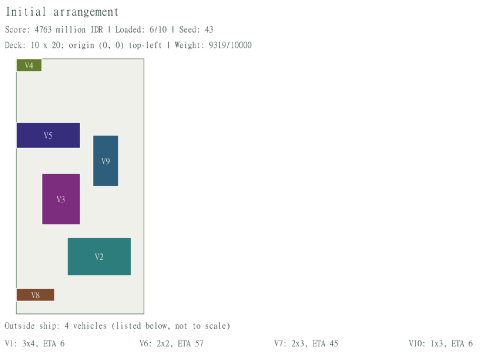
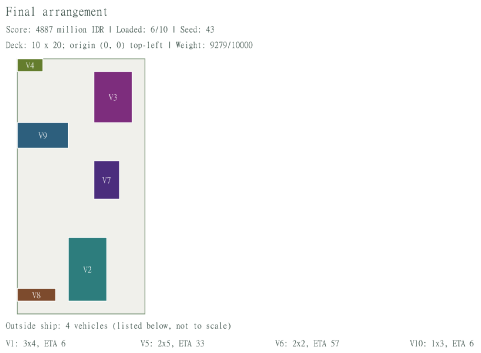


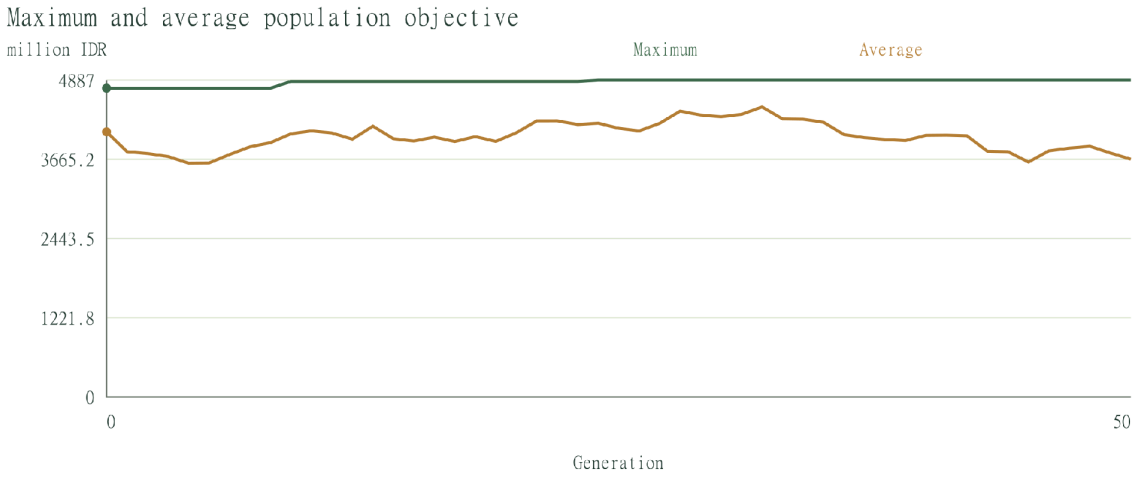
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 2: populasi 30, 50 generasi, seed 43

Objective 4.763 → 4.887; durasi 485,98 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: a06280b6-8021-426d-a915-9d92f4d5f2e7. Sumber: output/genetic_algorithm/a06280b6-8021-426d-a915-9d92f4d5f2e7/run.json.

State awal	State akhir
	

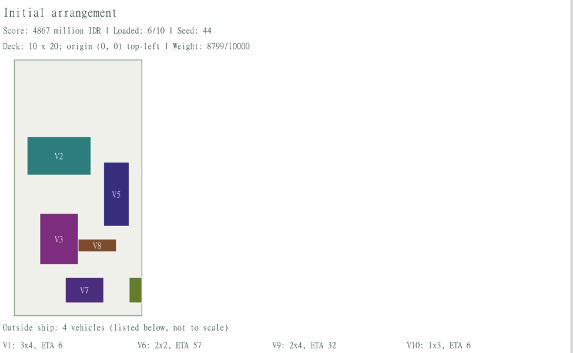
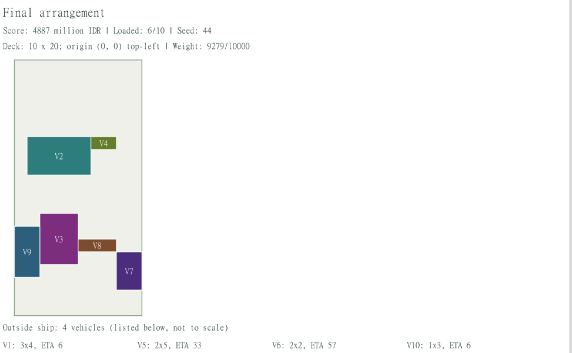


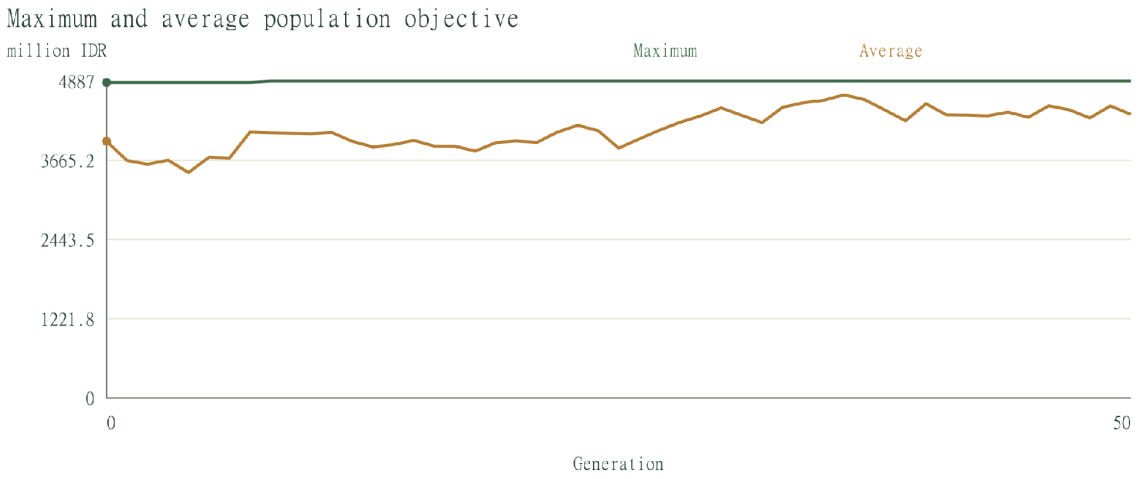
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 3: populasi 30, 50 generasi, seed 44

Objective 4.867 → 4.887; durasi 433,81 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: 78239e45-28ce-48ca-ab16-89c34ae106d7. Sumber: output/genetic_algorithm/78239e45-28ce-48ca-ab16-89c34ae106d7/run.json.

State awal	State akhir
	

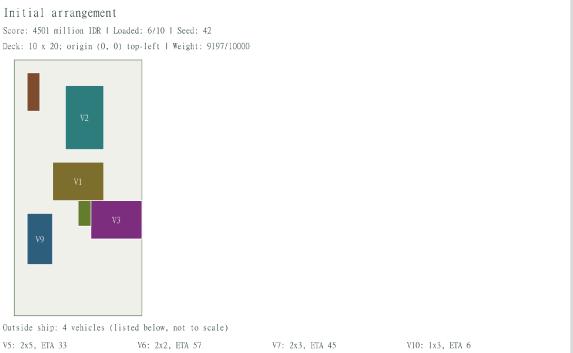
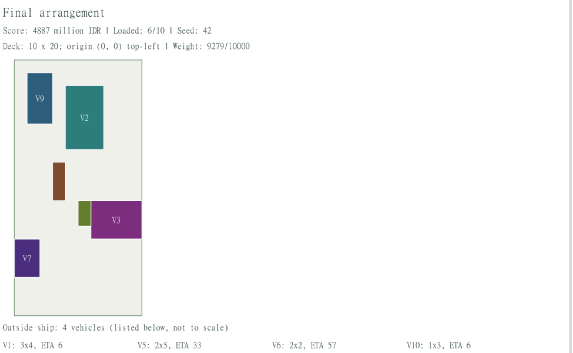


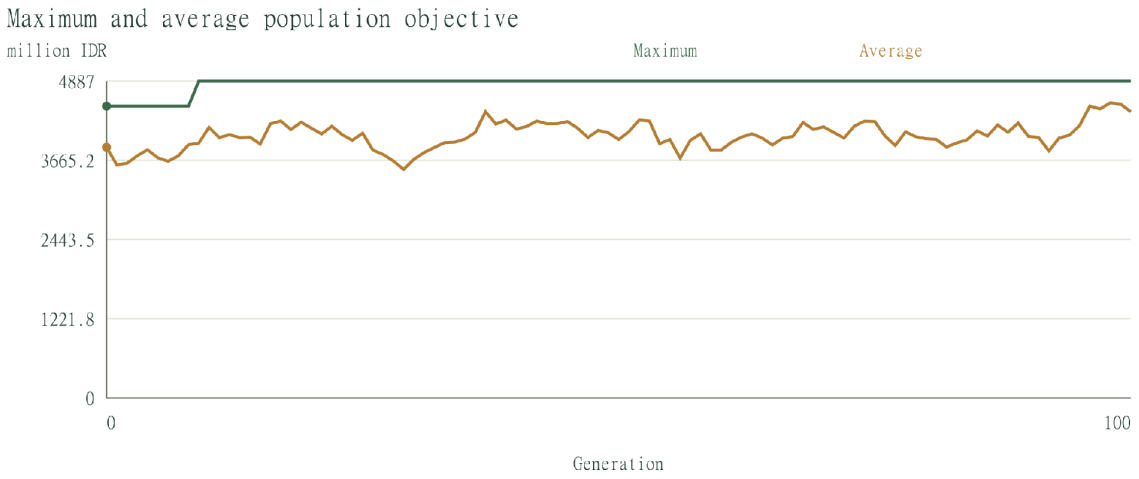
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 4: populasi 30, 100 generasi, seed 42

Objective 4.501 → 4.887; durasi 1.000,37 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: a7033ca3-cc08-42a7-a058-954640c3bca2. Sumber: output/genetic_algorithm/a7033ca3-cc08-42a7-a058-954640c3bca2/run.json.

State awal	State akhir
	

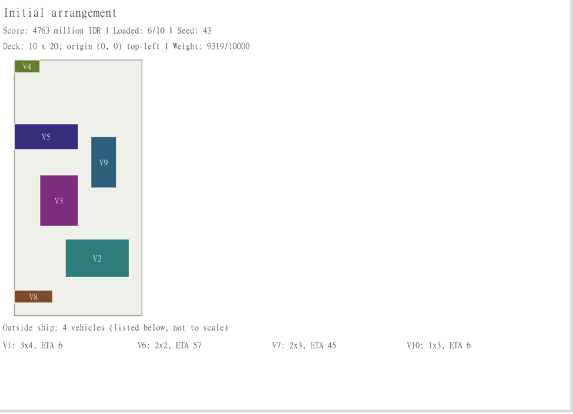
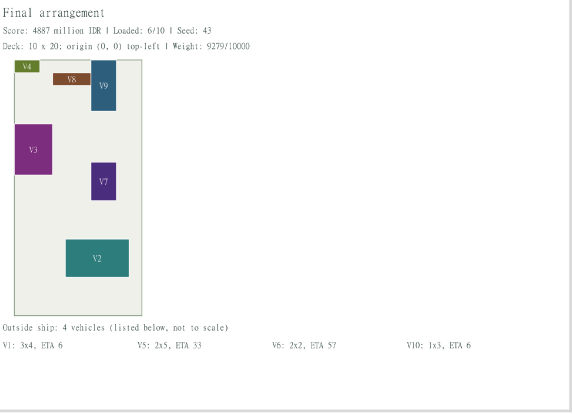


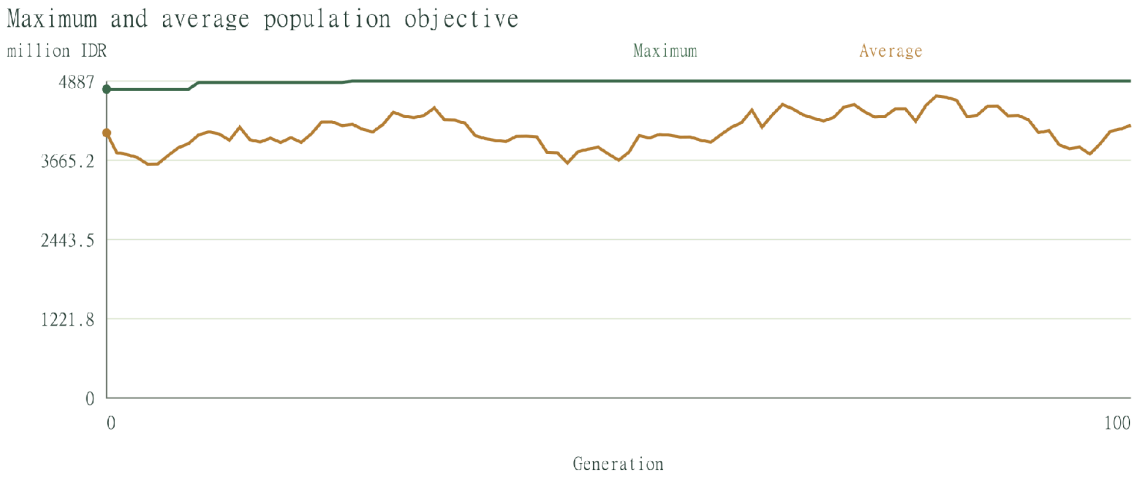
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 5: populasi 30, 100 generasi, seed 43

Objective 4.763 → 4.887; durasi 889,37 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: a2f2d212-137f-407d-99f0-1d3817b86aea. Sumber: output/genetic_algorithm/a2f2d212-137f-407d-99f0-1d3817b86aea/run.json.

State awal	State akhir
	

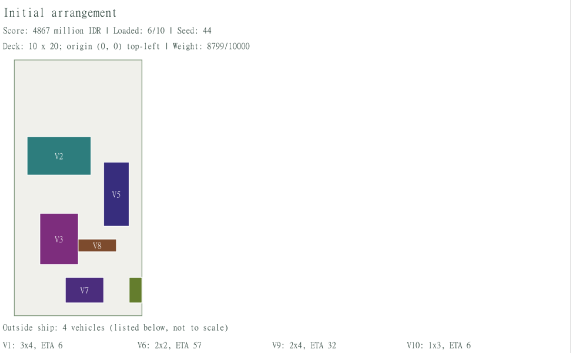


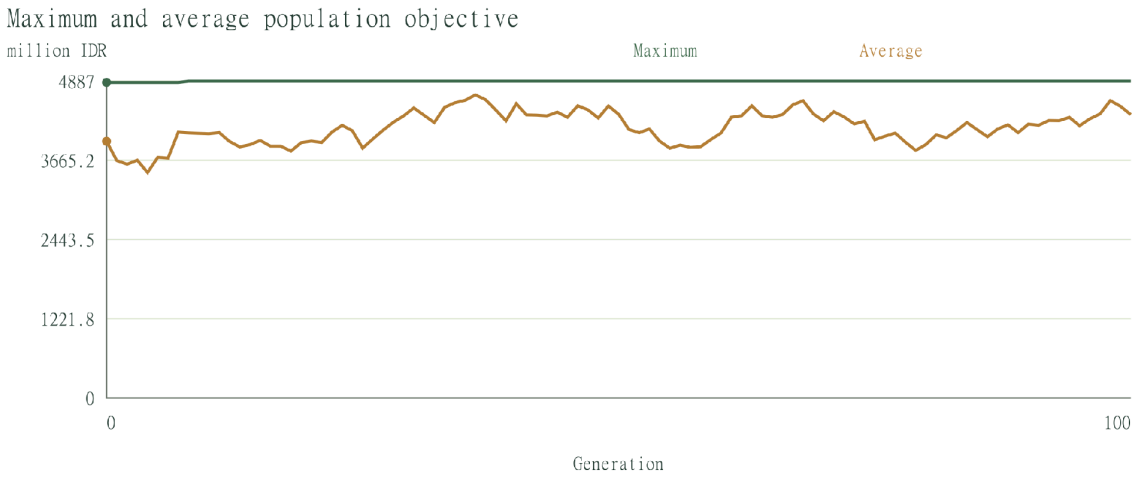
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 6: populasi 30, 100 generasi, seed 44

Objective 4.867 → 4.887; durasi 917,33 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: 49343eda-9dd9-4de8-a4b0-87d748ece0bf. Sumber: output/genetic_algorithm/49343eda-9dd9-4de8-a4b0-87d748ece0bf/run.json.

State awal	State akhir
	

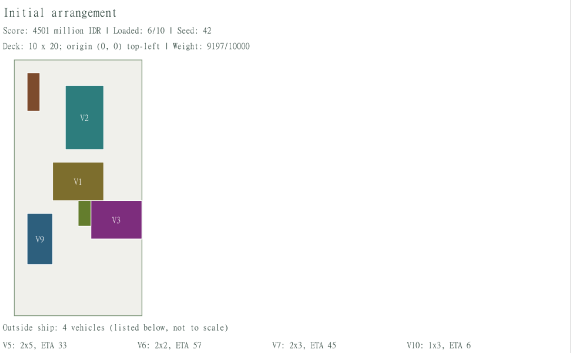
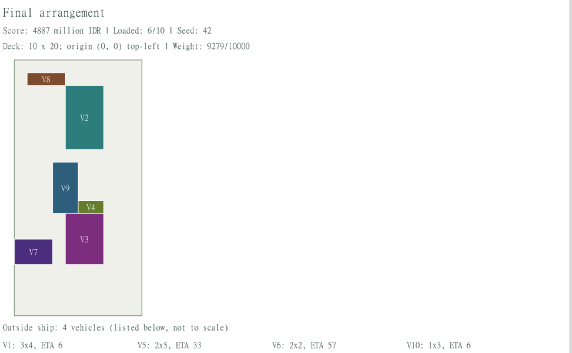


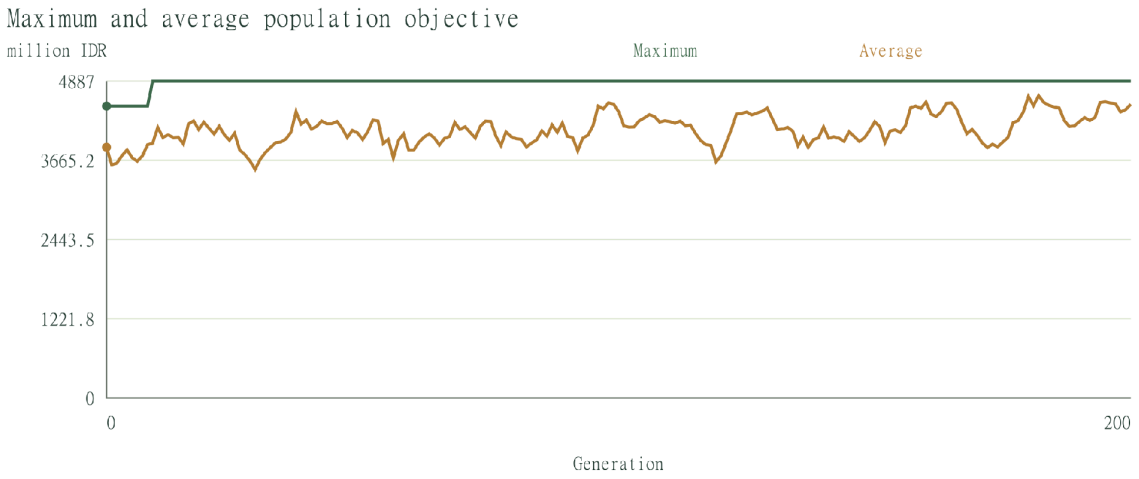
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 7: populasi 30, 200 generasi, seed 42

Objective 4.501 → 4.887; durasi 1.756,40 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: e438b021-6884-4ab1-a7dc-9aed2aefcf3f. Sumber: output/genetic_algorithm/e438b021-6884-4ab1-a7dc-9aed2aefcf3f/run.json.

State awal	State akhir
	

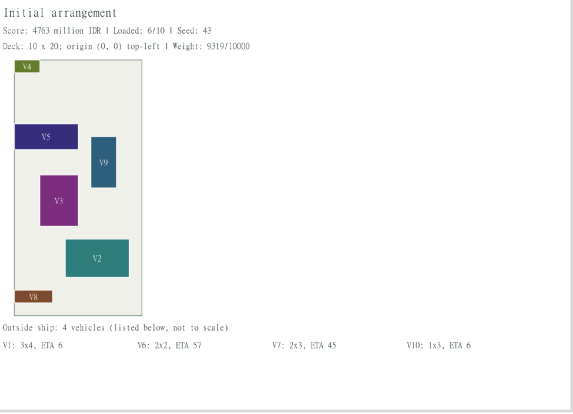


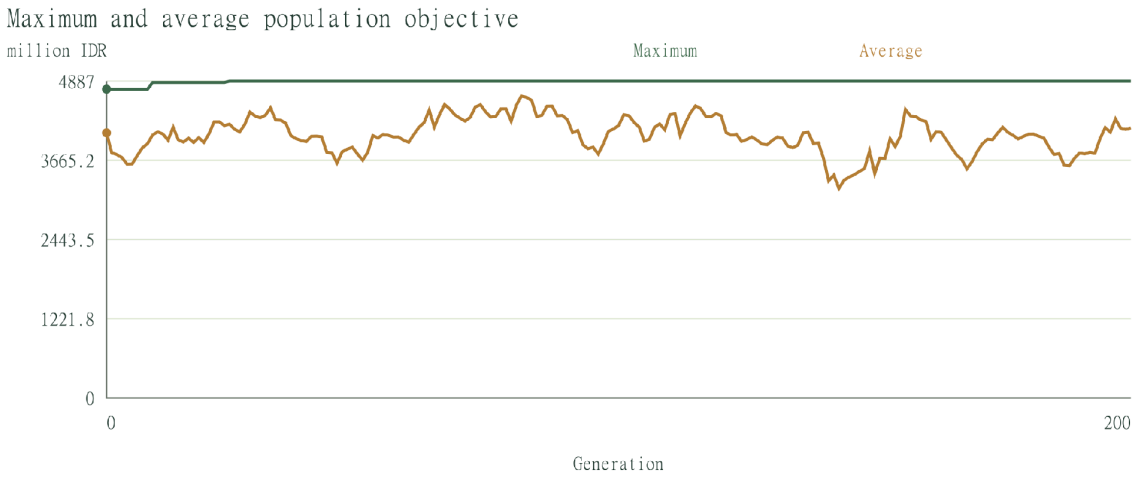
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 8: populasi 30, 200 generasi, seed 43

Objective 4.763 → 4.887; durasi 1.839,91 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: 756eddb5-b0f5-41ec-8dac-0203a4637aa8. Sumber: output/genetic_algorithm/756eddb5-b0f5-41ec-8dac-0203a4637aa8/run.json.

State awal	State akhir
	

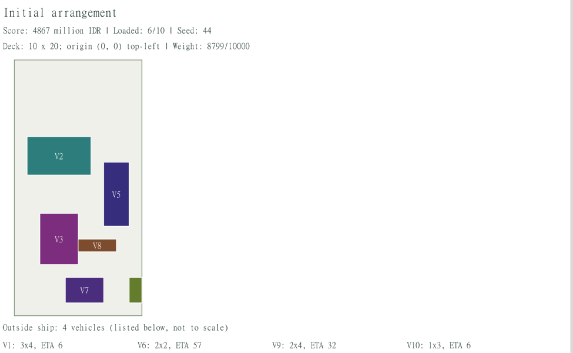
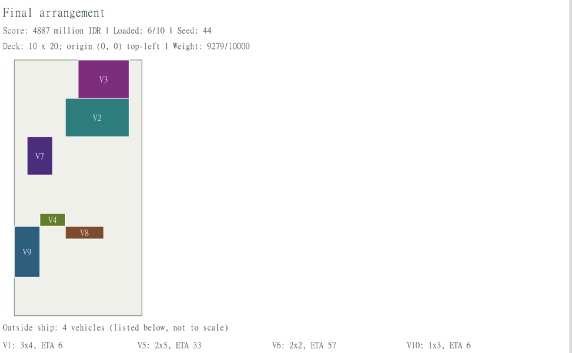


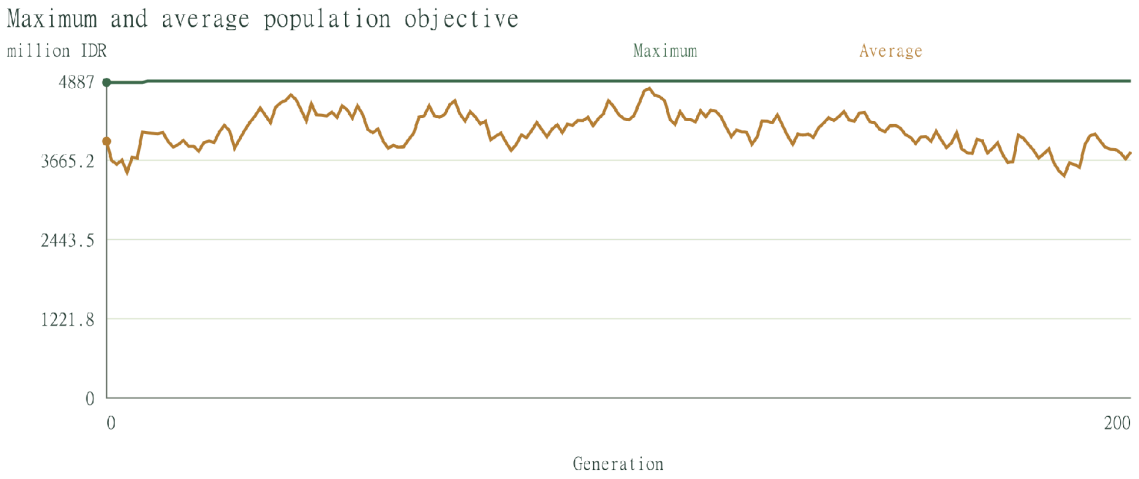
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 9: populasi 30, 200 generasi, seed 44

Objective 4.867 → 4.887; durasi 1.686,58 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: 8e6c263c-4532-4a27-bc8b-261b6577b604. Sumber: output/genetic_algorithm/8e6c263c-4532-4a27-bc8b-261b6577b604/run.json.

State awal	State akhir
	

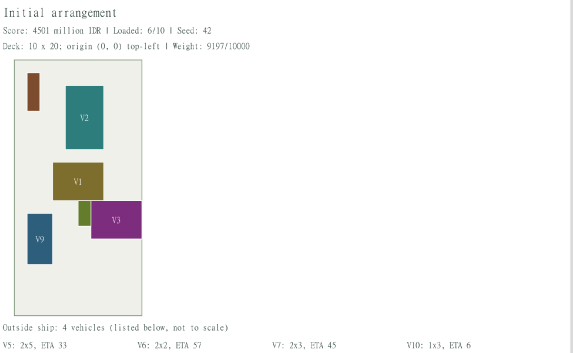
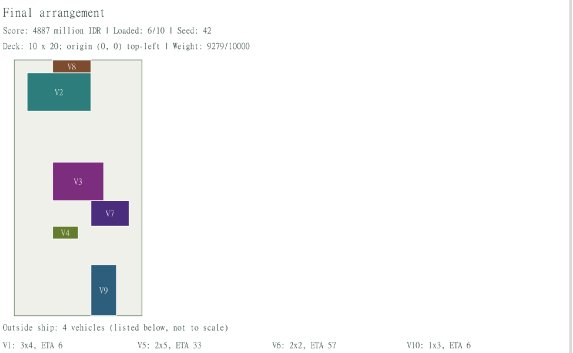


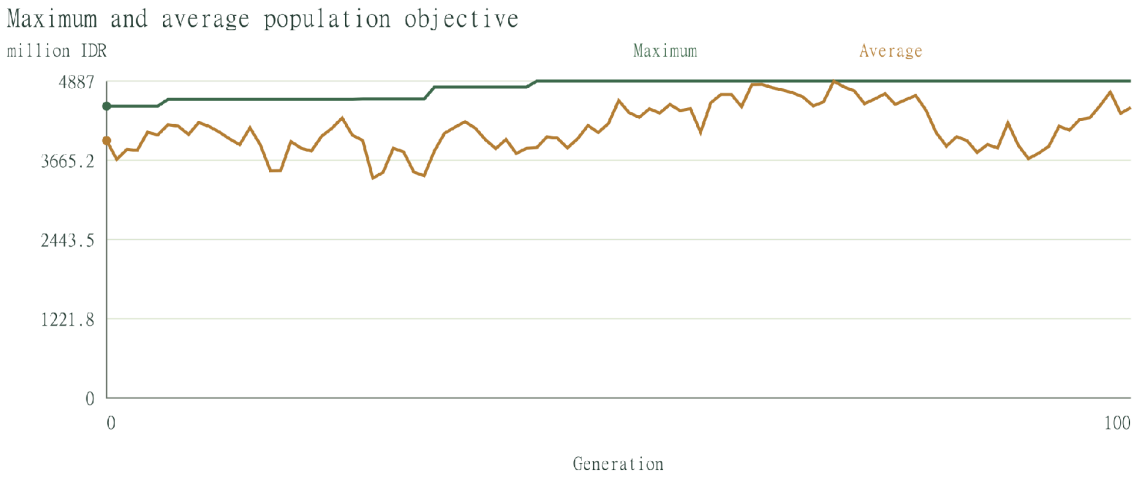
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 10: populasi 10, 100 generasi, seed 42

Objective 4.501 → 4.887; durasi 304,30 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: 816836a0-36de-4c96-b182-04606b613c29. Sumber: output/genetic_algorithm/816836a0-36de-4c96-b182-04606b613c29/run.json.

State awal	State akhir
	

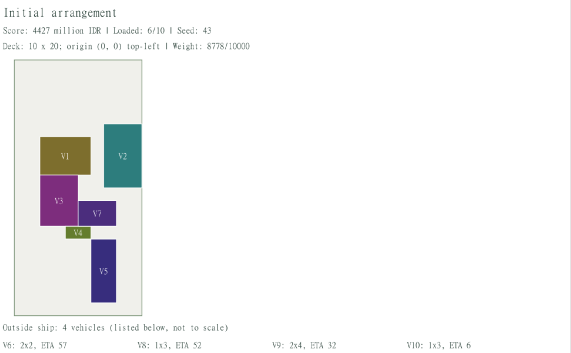
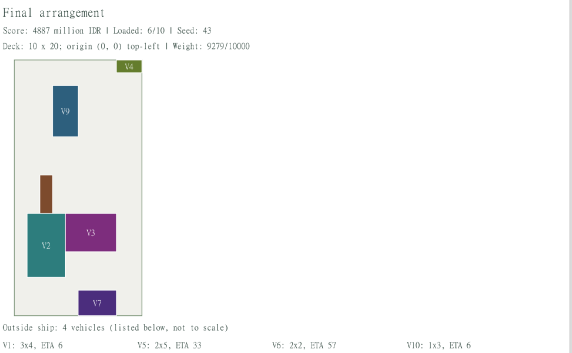


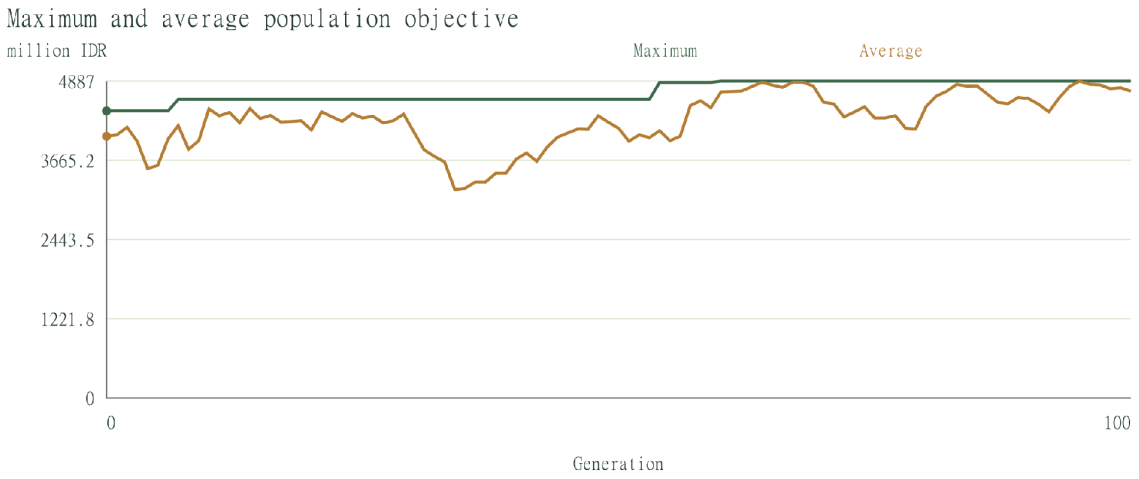
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 11: populasi 10, 100 generasi, seed 43

Objective 4.427 → 4.887; durasi 280,18 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: 33b7b353-ed57-4f76-953c-73d105afa323. Sumber: output/genetic_algorithm/33b7b353-ed57-4f76-953c-73d105afa323/run.json.

State awal	State akhir
	



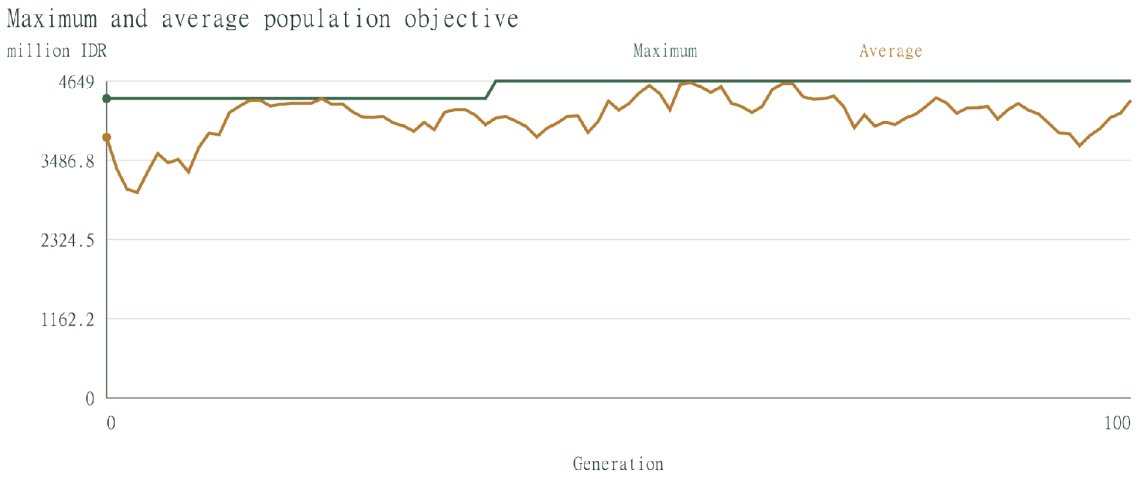
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 12: populasi 10, 100 generasi, seed 44

Objective 4.392 → 4.649; durasi 305,18 ms; 7 kendaraan dimuat, berat akhir 9.823/10.000, luas terpakai 49/200 grid.

Run ID: bb83e298-4bc2-4064-9332-2854324bbc3d. Sumber: output/genetic_algorithm/bb83e298-4bc2-4064-9332-2854324bbc3d/run.json.

State awal	State akhir
	

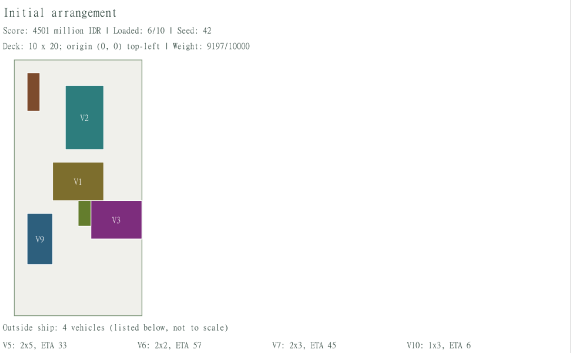
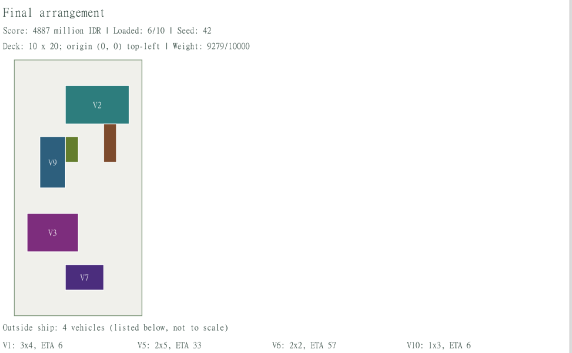


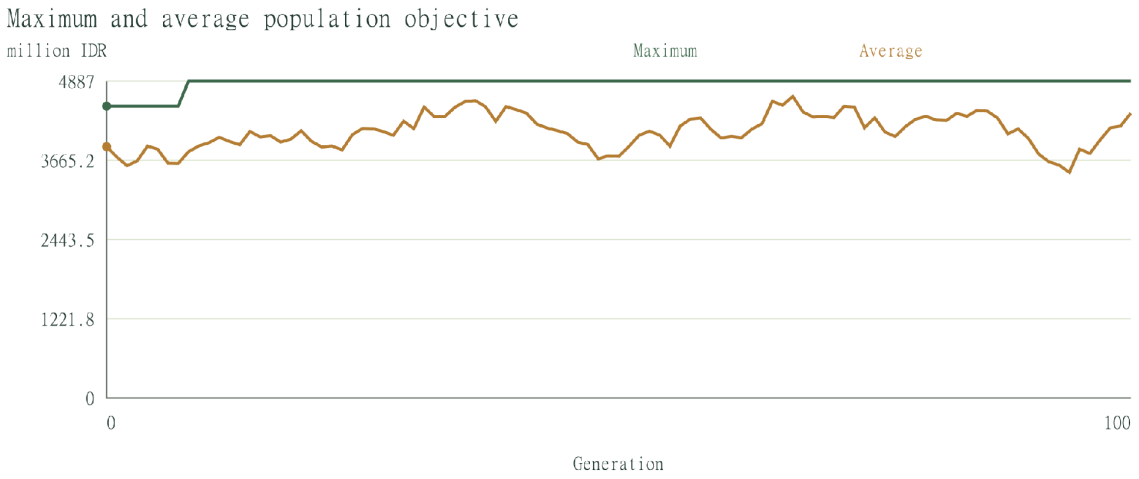
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 13: populasi 20, 100 generasi, seed 42

Objective 4.501 → 4.887; durasi 610,69 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: d8d95d92-d403-47b2-af85-956326dd1b26. Sumber: output/genetic_algorithm/d8d95d92-d403-47b2-af85-956326dd1b26/run.json.

State awal	State akhir
	

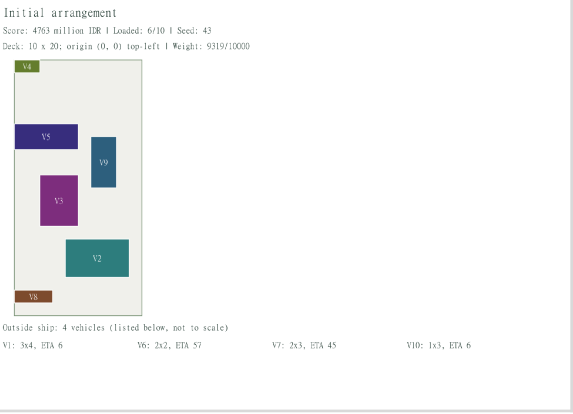
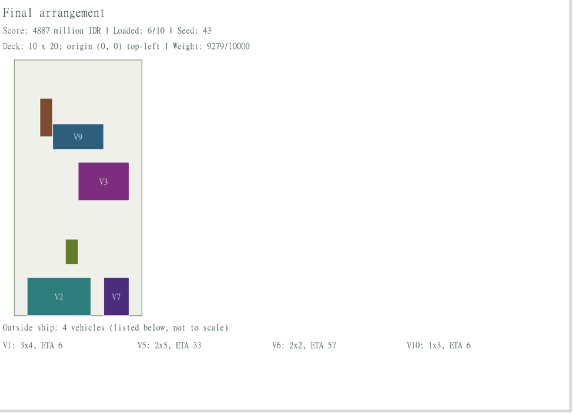


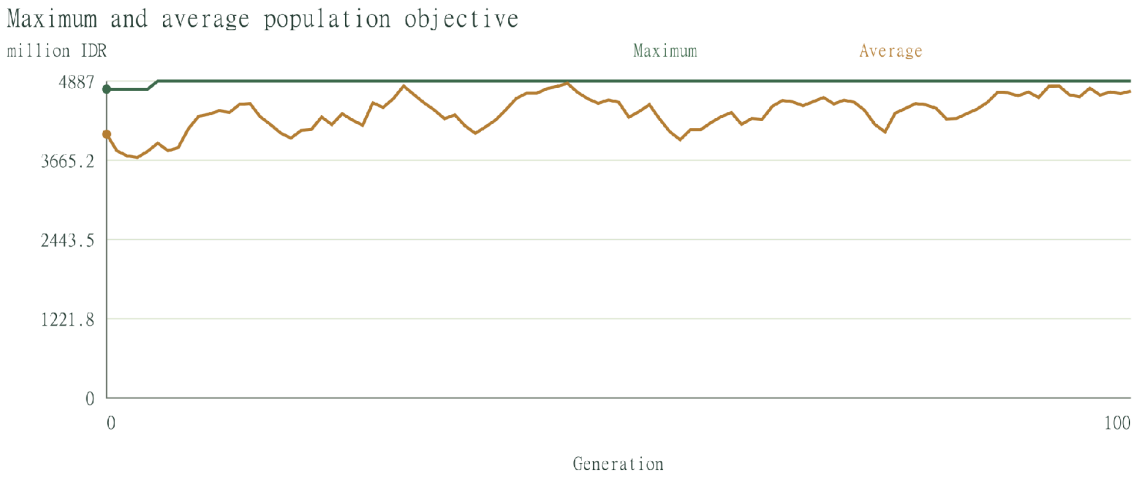
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 14: populasi 20, 100 generasi, seed 43

Objective 4.763 → 4.887; durasi 543,11 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: 5b2897f8-13ef-4e2b-a1de-c81aa19846f6. Sumber: output/genetic_algorithm/5b2897f8-13ef-4e2b-a1de-c81aa19846f6/run.json.

State awal	State akhir
	

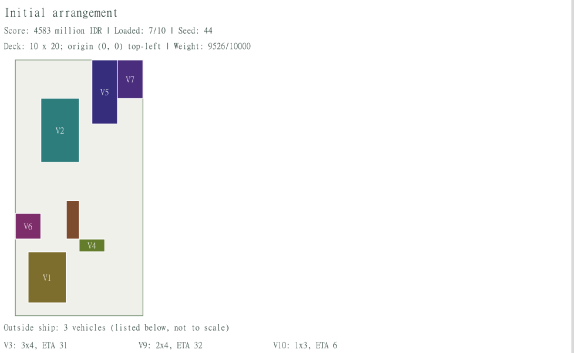
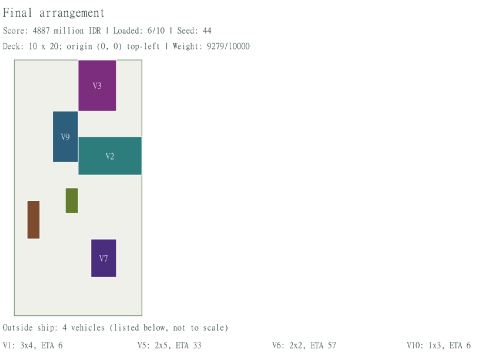


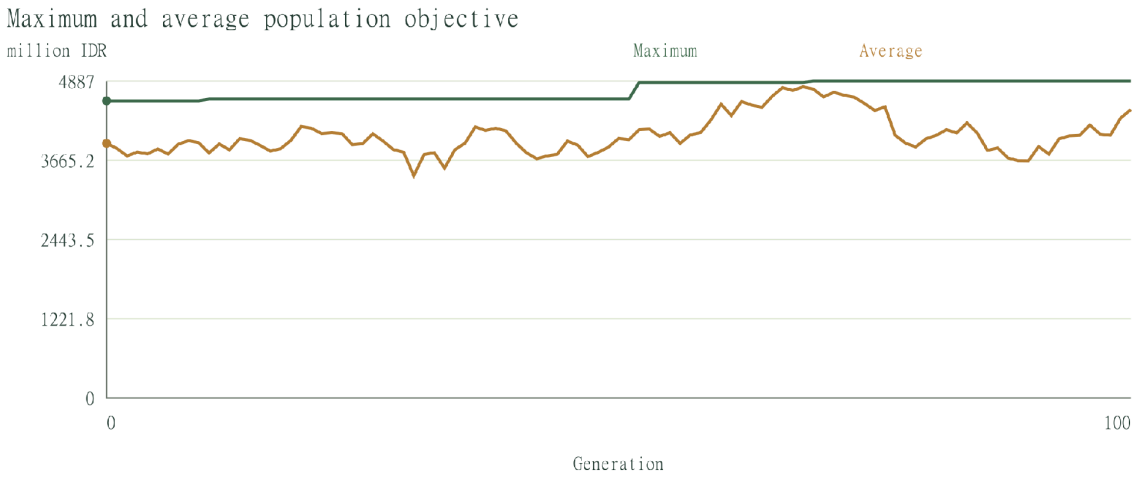
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 15: populasi 20, 100 generasi, seed 44

Objective 4.583 → 4.887; durasi 558,84 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: 261a889f-0d09-43dc-b863-38b9c1e44537. Sumber: output/genetic_algorithm/261a889f-0d09-43dc-b863-38b9c1e44537/run.json.

State awal	State akhir
	

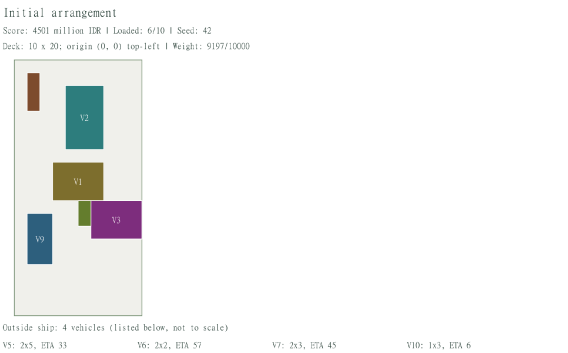
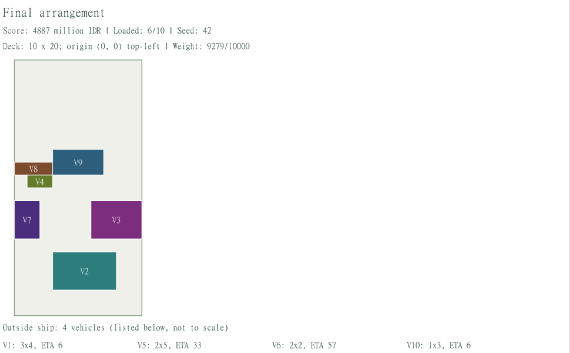


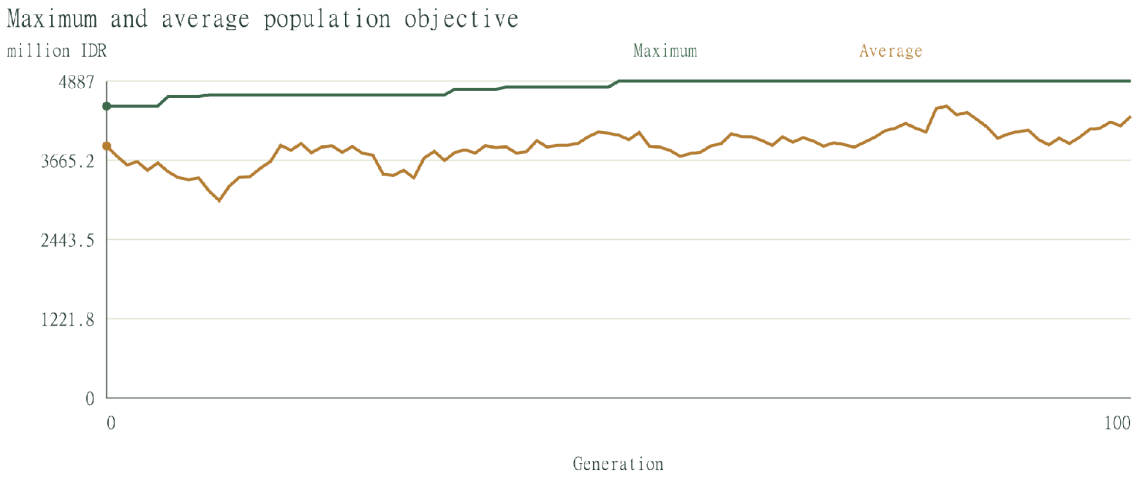
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 16: populasi 40, 100 generasi, seed 42

Objective 4.501 → 4.887; durasi 1.231,34 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: 823ba883-24ab-45dc-bf74-910b320d089c. Sumber: output/genetic_algorithm/823ba883-24ab-45dc-bf74-910b320d089c/run.json.

State awal	State akhir
	

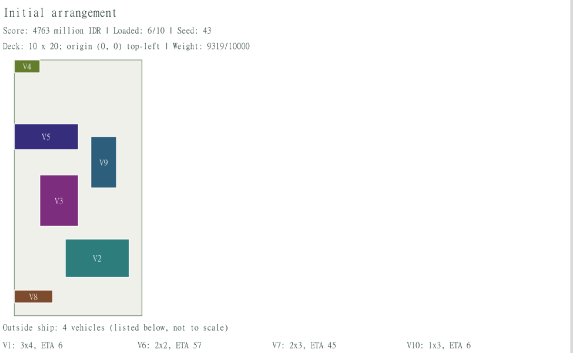
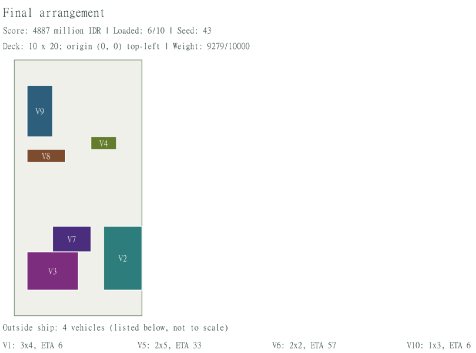


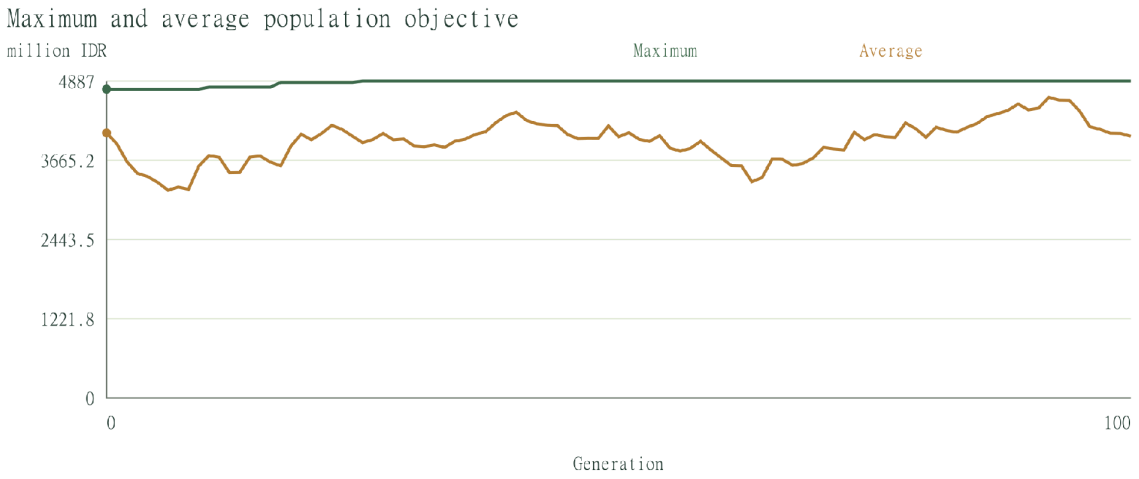
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 17: populasi 40, 100 generasi, seed 43

Objective 4.763 → 4.887; durasi 1.317,35 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: 114aa8d6-8fe8-4183-bdf2-e870b6474208. Sumber: output/genetic_algorithm/114aa8d6-8fe8-4183-bdf2-e870b6474208/run.json.

State awal	State akhir
	

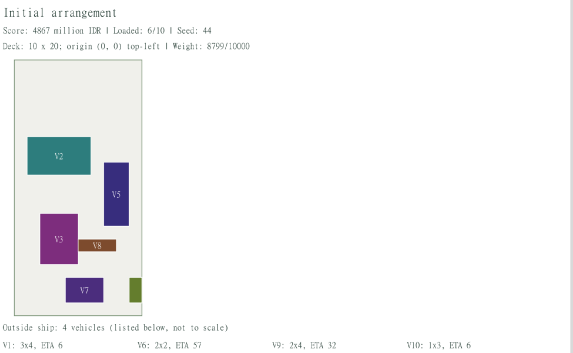
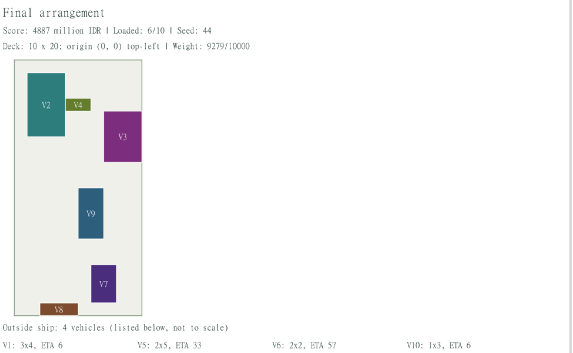


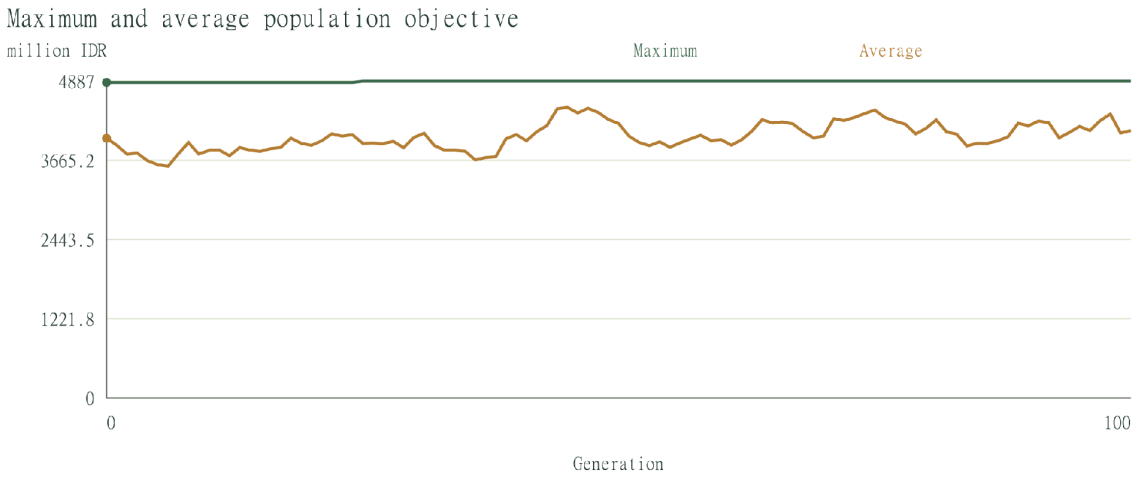
Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

Percobaan 18: populasi 40, 100 generasi, seed 44

Objective 4.867 → 4.887; durasi 1.264,70 ms; 6 kendaraan dimuat, berat akhir 9.279/10.000, luas terpakai 46/200 grid.

Run ID: b2565e0b-78a5-40ad-addf-5d55b22fb36d. Sumber: output/genetic_algorithm/b2565e0b-78a5-40ad-addf-5d55b22fb36d/run.json.

State awal	State akhir
	



Perkembangan fitness maksimum dan rata-rata populasi terhadap generasi.

2.6.7. Ringkasan hasil eksperimen

Pada konfigurasi yang diuji, populasi 30 telah mencapai optimum pada batas 50 generasi untuk seluruh seed, sehingga tambahan generasi terutama menambah waktu pencarian. Pada batas 100 generasi, populasi 20 merupakan ukuran terkecil yang mencapai optimum pada ketiga seed, sedangkan populasi 10 lebih cepat tetapi satu pengulangannya menghasilkan nilai lebih rendah. Pilihan parameter perlu mempertimbangkan kualitas solusi, konsistensi, dan waktu; hasil ini berlaku pada konfigurasi dan seed yang diuji.

2.7. Eksperimen Algoritma Simulated Annealing

Eksperimen dilakukan tiga kali menggunakan konfigurasi kapal dan kendaraan yang sama, dengan seed 42, 43, dan 44. Setiap seed menghasilkan initial state dan jalur pencarian yang berbeda.

Konfigurasi menggunakan geladak berukuran 10 × 20 grid, kapasitas berat 10.000 unit, dan 10 kendaraan. Objective yang digunakan adalah total shipping fee, sedangkan preserve_positions dinonaktifkan agar posisi awal dibangkitkan secara acak.

Parameter	Nilai
Maximum iterations	1000
Initial temperature	1000
Cooling rate	0.99
Minimum temperature	0.001
Stagnation window	100
Seed	42, 43, 44

1. Eksperimen 1

   Eksperimen pertama menggunakan seed 42.

   - State awal: Memuat enam kendaraan dengan total berat 8.991 unit dan objective 4.267.
   - State akhir: Memuat enam kendaraan dengan total berat 9.279 unit dan objective 4.887.

- Objective function: Meningkat sebesar 620, atau 14,53%. Nilai terbaik pertama kali ditemukan pada iterasi ke-357.
- Durasi proses pencarian: 240,60 milisekon.
- Iterasi hingga berhenti: 1.000 iterasi, dengan alasan iteration_limit.
- Frekuensi stagnasi: 3 episode.
- Penerimaan candidate lebih buruk: 27 dari 374 candidate dengan $\Delta E < 0$.

Pada awal pencarian, current objective berfluktuasi hingga mencapai nilai minimum 2.009. Hal ini menunjukkan bahwa algoritma menerima penurunan objective selama eksplorasi. Meskipun demikian, best objective tetap disimpan dan akhirnya mencapai 4.887.

2. Eksperimen 2

   Eksperimen kedua menggunakan seed 43.

   - State awal: Memuat enam kendaraan dengan total berat 9.614 unit dan objective 3.724.
   - State akhir: Memuat enam kendaraan dengan total berat 9.279 unit dan objective 4.887.
   - Objective function: Meningkat sebesar 1.163, atau 31,23%. Nilai terbaik pertama kali ditemukan pada iterasi ke-79.
   - Durasi proses pencarian: 203,29 milisekon.
   - Iterasi hingga berhenti: 1.000 iterasi, dengan alasan iteration_limit.
   - Frekuensi stagnasi: 1 episode.
   - Penerimaan candidate lebih buruk: 16 dari 417 candidate dengan $\Delta E < 0$.

   Eksperimen ini menemukan solusi terbaik lebih cepat daripada eksperimen pertama. Berat total hasil akhir juga lebih rendah daripada state awal, tetapi objective meningkat. Dengan demikian, memaksimalkan shipping fee tidak selalu berarti memaksimalkan berat muatan.

3. Eksperimen 3

   Eksperimen ketiga menggunakan seed 44.

   - State awal: Memuat enam kendaraan dengan total berat 9.005 unit dan objective 3.299.
   - State akhir: Memuat enam kendaraan dengan total berat 9.279 unit dan objective 4.887.
   - Objective function: Meningkat sebesar 1.588, atau 48,14%. Nilai terbaik pertama kali ditemukan pada iterasi ke-75.
   - Durasi proses pencarian: 206,70 milisekon.
   - Iterasi hingga berhenti: 1.000 iterasi, dengan alasan iteration_limit.
   - Frekuensi stagnasi: 1 episode.
   - Penerimaan candidate lebih buruk: 24 dari 395 candidate dengan $\Delta E < 0$.

   Eksperimen ketiga memiliki objective awal terendah, tetapi paling cepat menemukan nilai terbaik. Setelah mencapai 4.887, current objective masih sempat menurun karena temperatur memungkinkan penerimaan candidate yang lebih buruk. Solusi terbaik tetap dipertahankan sebagai hasil akhir.

2.7.1. Visualisasi Hasil Eksperimen

Ketiga percobaan menghasilkan pilihan kendaraan akhir yang sama, yaitu V2, V3, V4, V7, V8, dan V9, tetapi posisi dan orientasinya dapat berbeda. Penomoran kendaraan mengikuti urutannya pada input/config.json.

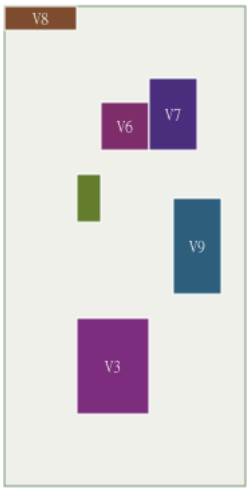
State Awal dan Akhir Eksperimen dapat dilihat pada gambar-gambar berikut:

Eksperimen 1 | Seed 42 | Initial (kiri) dan Final (kanan)

Initial arrangement

Score: 4267 million IDR | Loaded: 6/10 | Seed: 42

Deck: 10 x 20; origin (0, 0) top-left | Weight: 8991/10000



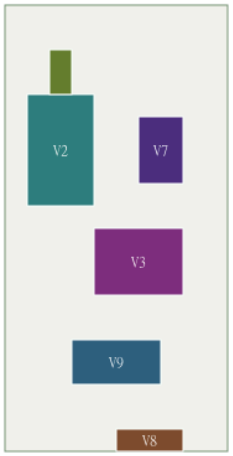
Outside ship: 4 vehicles (listed below, not to scale)

V1: 3x4, ETA 6 V2: 3x5, ETA 20 V5: 2x5, ETA 33 V10: 1x3, ETA 6

Final arrangement

Score: 4887 million IDR | Loaded: 6/10 | Seed: 42

Deck: 10 x 20; origin (0, 0) top-left | Weight: 9279/10000



Outside ship: 4 vehicles (listed below, not to scale)

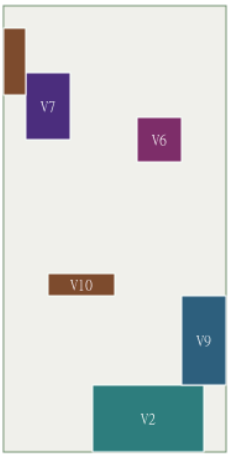
V1: 3x4, ETA 6 V5: 2x5, ETA 33 V6: 2x2, ETA 57 V10: 1x3, ETA 6

Eksperimen 2 | Seed 43 | Initial (kiri) dan Final (kanan)

Initial arrangement

Score: 3724 million IDR | Loaded: 6/10 | Seed: 43

Deck: 10 x 20; origin (0, 0) top-left | Weight: 9614/10000



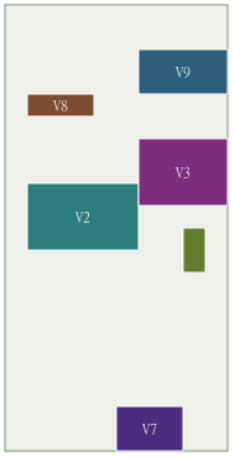
Outside ship: 4 vehicles (listed below, not to scale)

V1: 3x4, ETA 6 V3: 3x4, ETA 31 V4: 1x2, ETA 60 V5: 2x5, ETA 33

Final arrangement

Score: 4887 million IDR | Loaded: 6/10 | Seed: 43

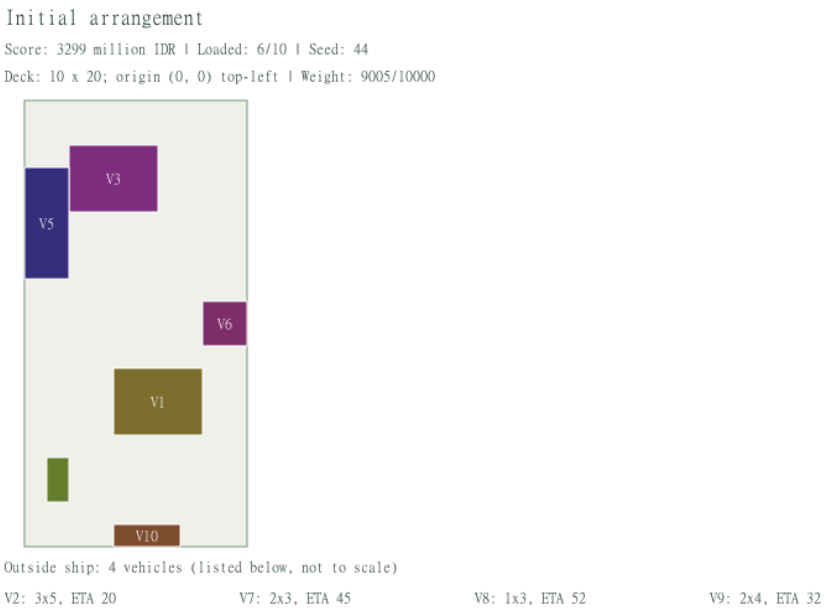
Deck: 10 x 20; origin (0, 0) top-left | Weight: 9279/10000

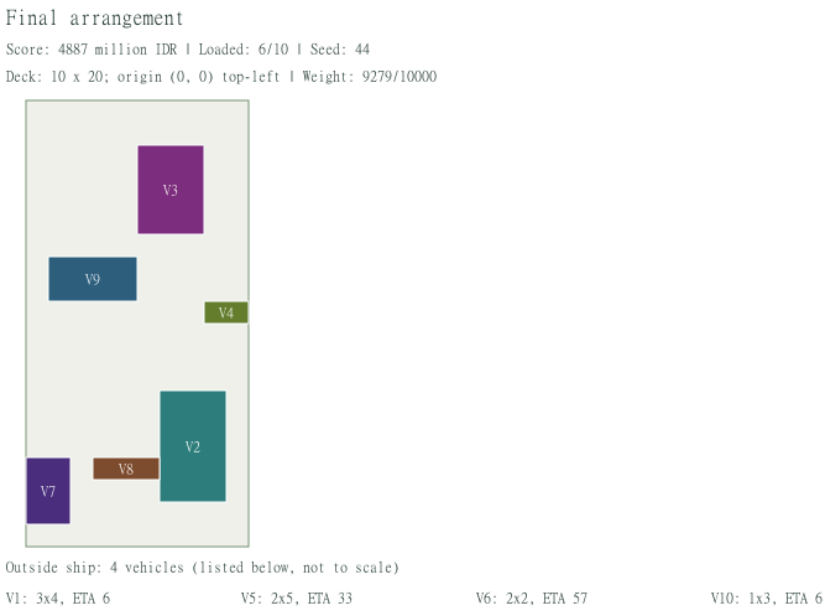


Outside ship: 4 vehicles (listed below, not to scale)

V1: 3x4, ETA 6 V5: 2x5, ETA 33 V6: 2x2, ETA 57 V10: 1x3, ETA 6

Eksperimen 3 | Seed 44 | Initial (kiri) dan Final (kanan)

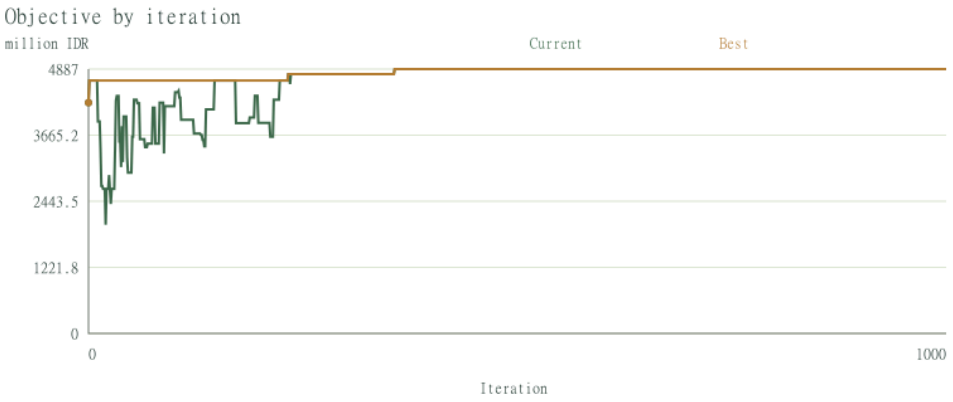




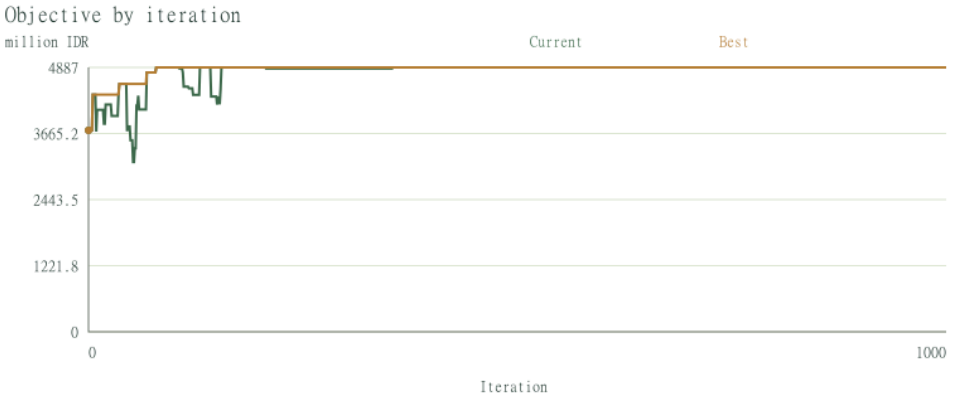
Objective Function terhadap iterasi

Grafik berikut memperlihatkan current objective dan best objective. Current objective dapat menurun karena penerimaan solusi yang lebih buruk, sedangkan best objective tidak menurun.

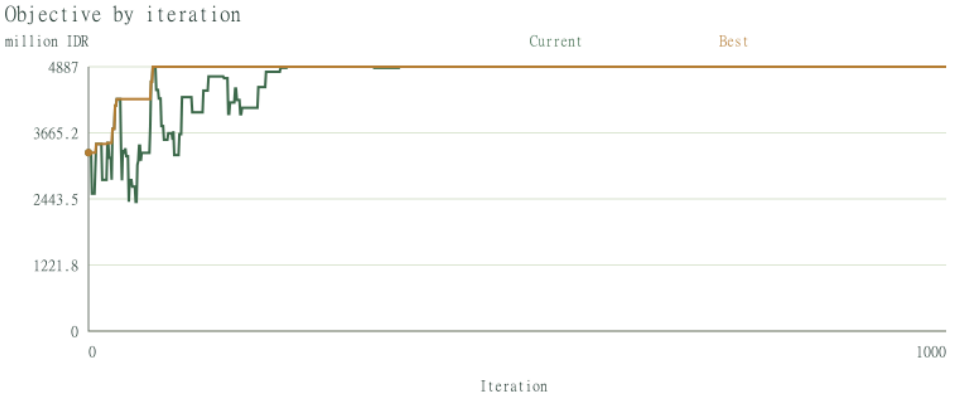
Eksperimen 1 | Seed 42



Eksperimen 2 | Seed 43

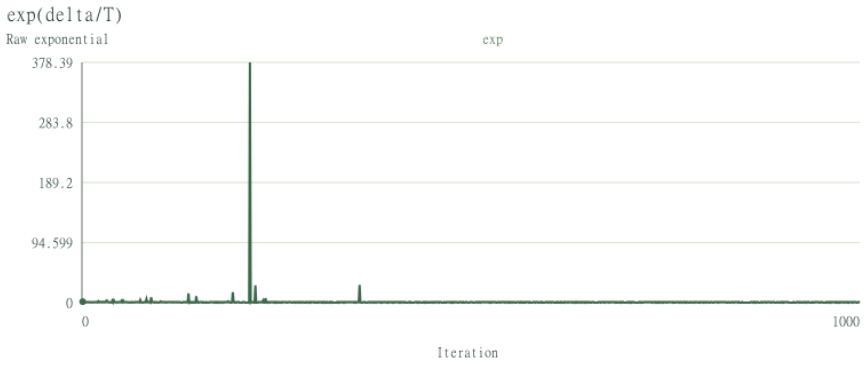


Eksperimen 3 | Seed 44

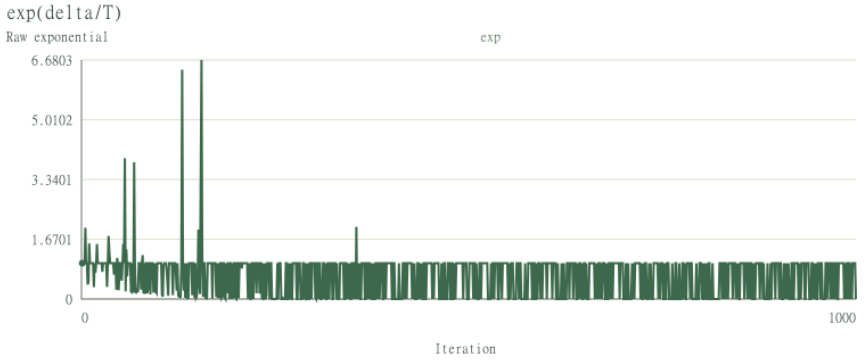


Grafik $e^{\Delta E/T}$

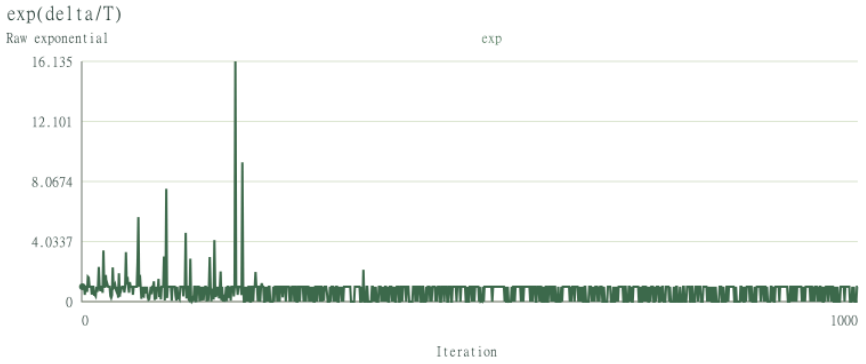
1. Eksperimen 1



2. Eksperimen 2



3. Eksperimen 3



Nilai eksponensial lebih besar dari satu menunjukkan candidate dengan $\Delta E > 0$, yang selalu diterima. Untuk candidate dengan $\Delta E < 0$, nilainya berada di antara nol dan satu serta menjadi probabilitas penerimaan. Candidate dengan $\Delta E = 0$ menghasilkan nilai satu.

Ketika temperatur semakin rendah, nilai eksponensial untuk penurunan objective mendekati nol. Namun, keseluruhan grafik tidak harus menurun karena ΔE berubah pada setiap iterasi. Tidak ditemukan overflow pada ketiga percobaan ini.

2.7.2. Ringkasan Hasil

Percobaan	Seed	Objective awal	Objective akhir	Peningkatan	Iterasi pertama mencapai nilai terbaik	Durasi
1	42	4.267	4.887	14,53%	357	240,60 ms
2	43	3.724	4.887	31,23%	79	203,29 ms
3	44	3.299	4.887	48,14%	75	206,70 ms

Ketiga percobaan menjalankan 1.000 iterasi dan berhenti karena batas iterasi. Rata-rata objective akhir adalah 4.887, dengan simpangan baku 0, sedangkan rata-rata durasi pencarian adalah 216,86 milisekon.

Jumlah episode stagnasi berturut-turut adalah 3, 1, dan 1. Stagnasi berarti tidak ada perbaikan best objective selama 100 iterasi, bukan bukti bahwa current state berada pada local optimum. Setelah solusi optimum ditemukan, episode stagnasi tetap dapat tercatat karena tidak ada lagi nilai yang lebih tinggi.

2.8. Analisis Hasil Eksperimen

Perbandingan dilakukan pada konfigurasi yang sama, yaitu geladak 10 × 20 grid, kapasitas berat 10.000 unit, dan 10 kendaraan, dengan seed 42, 43, dan 44. Objective berupa total shipping fee dalam juta rupiah. Nilai optimum global konfigurasi ini adalah 4.887, dibuktikan melalui batas atas seluruh 1.024 subset kendaraan berdasarkan kapasitas berat serta susunan valid yang mencapai nilai tersebut.

Algoritma	Hasil seed 42 / 43 / 44	Rata-rata objective	Rata-rata waktu (ms)	Optimum
HC Steepest Ascent	4.887 / 4.887 / 4.483	4.752,33	459,42	2/3
HC Stochastic	4.887 / 4.887 / 4.483	4.752,33	661,53	2/3
HC Sideways	4.887 / 4.887 / 4.483	4.752,33	1.729,30	2/3

HC Random Restart	4.887 / 4.887 / 4.887	4.887,00	1.543,36	3/3
Simulated Annealing	4.887 / 4.887 / 4.887	4.887,00	216,86	3/3
GA P30 G50	4.887 / 4.887 / 4.887	4.887,00	479,32	3/3
GA P30 G100	4.887 / 4.887 / 4.887	4.887,00	935,69	3/3
GA P30 G200	4.887 / 4.887 / 4.887	4.887,00	1.760,96	3/3
GA P10 G100	4.887 / 4.887 / 4.649	4.807,67	296,55	2/3
GA P20 G100	4.887 / 4.887 / 4.887	4.887,00	570,88	3/3
GA P40 G100	4.887 / 4.887 / 4.887	4.887,00	1.271,13	3/3

P adalah ukuran populasi dan G adalah batas generasi GA. Kolom optimum menunjukkan jumlah run yang mencapai 4.887 dari tiga pengulangan. HC dan SA memakai state awal yang sama untuk seed yang sama; state awal GA merupakan individu terbaik dari populasi awalnya.

- Kualitas dan konsistensi solusi

  SA dan HC Random Restart mencapai optimum pada ketiga seed. HC Steepest Ascent, Stochastic, dan Sideways menghasilkan nilai yang sama, yaitu 4.887, 4.887, dan 4.483. Pada seed 44, hasilnya masih 404 atau 8,27% di bawah optimum. Sideways menambah pencarian pada plateau tanpa memperbaiki hasil, sedangkan restart memperoleh hasil optimal melalui titik awal baru. GA mencapai optimum pada 17 dari 18 run; satu pengecualian adalah populasi 10 dengan seed 44 yang menghasilkan 4.649. Ini menunjukkan bahwa eksplorasi melalui restart, penerimaan solusi lebih buruk pada SA, atau populasi GA dapat membantu pencarian keluar dari wilayah solusi yang kurang baik.

- Durasi pencarian

  SA mencatat rata-rata waktu terendah, yaitu 216,86 ms, sekaligus mencapai optimum pada seluruh seed. HC Steepest Ascent membutuhkan 459,42 ms, sedangkan GA populasi 30 dengan 50 generasi membutuhkan 479,32 ms dan konsisten optimal. HC Sideways memerlukan 1.729,30 ms tanpa meningkatkan hasil dibandingkan Steepest Ascent. Jumlah iterasi tidak dapat dibandingkan

langsung: HC memindai seluruh neighborhood, SA mengambil satu neighbor valid, sedangkan GA memproses satu populasi per generasi. Timer GA juga mencakup pembangkitan sisa populasi, sehingga waktu dibaca sebagai hasil implementasi ini, bukan perbandingan biaya komputasi yang sepenuhnya seragam.

- Pengaruh parameter dan hasil keseluruhan

  Pada populasi GA 30, batas 50, 100, dan 200 generasi menghasilkan objective akhir yang sama, sementara waktu meningkat. Pada 100 generasi, populasi 20, 30, dan 40 konsisten optimal, sedangkan populasi 10 lebih cepat tetapi satu run belum optimal. Untuk konfigurasi dan parameter yang diuji, SA memberikan gabungan kualitas, konsistensi, dan waktu terbaik. GA populasi 30 dengan 50 generasi menjadi alternatif yang konsisten. Kesimpulan ini terbatas pada satu konfigurasi dan tiga seed per kombinasi; hasilnya belum menunjukkan keunggulan umum pada persoalan yang lebih besar.

BAB III KESIMPULAN DAN SARAN

3.1.Simpulan

Berdasarkan implementasi dan hasil eksperimen yang telah dilakukan, dapat disimpulkan bahwa algoritma Hill Climbing Steepest Ascent, Genetic Algorithm, dan Simulated Annealing dapat digunakan untuk menyelesaikan persoalan optimasi penataan kendaraan pada kapal. Ketiga algoritma mampu menghasilkan konfigurasi penempatan kendaraan yang valid dengan memperhatikan batas dimensi geladak, kapasitas berat maksimum, serta larangan tumpang tindih antar kendaraan. Namun, terdapat perbedaan dalam kualitas solusi, konsistensi hasil, dan waktu pencarian yang dibutuhkan.

Hasil eksperimen menunjukkan bahwa Simulated Annealing memberikan performa terbaik pada konfigurasi yang diuji. Algoritma ini berhasil mencapai nilai objektif optimum global sebesar 4.887 pada ketiga seed dengan rata-rata waktu pencarian 216,86 milisekon. Sementara itu, Hill Climbing Steepest Ascent memperoleh rata-rata nilai objektif 4.752,33 dengan waktu pencarian 459,42 milisekon. Algoritma tersebut hanya mencapai optimum pada dua dari tiga percobaan karena pencarian dapat berhenti ketika tidak ditemukan solusi tetangga yang lebih baik. Genetic Algorithm juga menunjukkan hasil yang konsisten, khususnya pada populasi 30 dengan 50 generasi, yang berhasil mencapai optimum pada seluruh percobaan dengan rata-rata waktu 479,32 milisekon.

Eksperimen juga menunjukkan bahwa penambahan jumlah generasi atau ukuran populasi pada Genetic Algorithm tidak selalu meningkatkan kualitas solusi, tetapi dapat menambah waktu komputasi. Selain itu, mekanisme eksplorasi seperti penerimaan solusi yang lebih buruk pada Simulated Annealing, penggunaan populasi pada Genetic Algorithm, dan Random Restart pada Hill Climbing dapat membantu pencarian menghindari keterbatasan optimum lokal.

Dengan demikian, Simulated Annealing menjadi algoritma yang paling efektif berdasarkan kombinasi kualitas solusi, konsistensi, dan waktu pencarian pada

eksperimen ini. Meskipun demikian, hasil tersebut masih terbatas pada satu konfigurasi persoalan dan tiga seed pengujian sehingga belum dapat digeneralisasikan untuk seluruh kondisi penataan kendaraan.

3.2.Saran

Berdasarkan hasil implementasi dan eksperimen, terdapat beberapa saran yang dapat dipertimbangkan untuk pengembangan lebih lanjut.

1. Memperluas variasi konfigurasi pengujian. Eksperimen selanjutnya dapat menggunakan jumlah kendaraan, dimensi geladak, dan kapasitas berat kapal yang lebih beragam untuk mengevaluasi kemampuan setiap algoritma dalam menyelesaikan persoalan dengan tingkat kompleksitas berbeda.
2. Melakukan pengujian dengan lebih banyak seed. Penambahan jumlah percobaan dengan seed yang berbeda diperlukan agar konsistensi dan kestabilan hasil setiap algoritma dapat dianalisis secara lebih representatif.
3. Mengoptimalkan parameter algoritma. Pengujian lanjutan dapat dilakukan terhadap temperatur awal dan *cooling rate* pada *Simulated Annealing*, ukuran populasi serta probabilitas *crossover* dan mutasi pada *Genetic Algorithm*, serta strategi pencarian tetangga pada *Hill Climbing*. Hal ini bertujuan untuk memperoleh keseimbangan yang lebih baik antara kualitas solusi dan waktu komputasi.
4. Mengembangkan strategi pencarian yang lebih efisien. Implementasi dapat ditingkatkan melalui optimasi pembangkitan solusi tetangga, pengurangan evaluasi yang tidak diperlukan, serta pengembangan mekanisme eksplorasi untuk mengurangi risiko terjebak pada optimum lokal.
5. Mengembangkan fungsi objektif yang lebih komprehensif. Selain memaksimalkan total tarif pengiriman dan menggunakan prioritas ETA secara terpisah, pengembangan selanjutnya dapat mempertimbangkan pendekatan *multi-objective optimization* yang menggabungkan aspek pendapatan, prioritas kedatangan kendaraan, dan efisiensi pemanfaatan ruang geladak.

BAB IV Bonus

3.3. Bonus No. 1 Penambahan Objective Function

Selain memaksimalkan *shipping fee*, program menyediakan fungsi objektif alternatif yang mengutamakan kendaraan dengan ETA lebih kecil. Fungsi ini hanya menggunakan atribut ETA yang sudah tersedia, tanpa menambahkan variabel baru pada kendaraan.

Fungsi objektifnya dirumuskan sebagai:

$$f_{ETA}(s) = \sum_{v \in L(s)} (E_{\max} + 1 - ETA(v))$$

Dengan:

- ***L*(*s*)**: himpunan kendaraan yang dimuat pada *state s*.
- ***E*max**: ETA terbesar dari seluruh kendaraan dalam konfigurasi, termasuk yang tidak dimuat.
- **ETA(*v*)**: ETA kendaraan *v*.

Kendaraan dengan ETA lebih kecil memperoleh poin lebih tinggi. Penambahan konstanta 1 memastikan setiap kendaraan memiliki kontribusi positif, sedangkan kapal kosong memiliki skor 0. Sebagai contoh, jika ETA maksimum adalah 5, kendaraan dengan ETA 0, 2, dan 5 masing-masing memperoleh 6, 4, dan 1 poin.

Objective ini memaksimalkan total poin prioritas kendaraan yang dimuat, bukan meminimalkan rata-rata ETA. *Shipping fee* tidak ikut dihitung. Seluruh *constraint* penempatan dan kapasitas berat tetap berlaku. Mode ini dapat digunakan oleh seluruh varian HC, SA, dan GA melalui pilihan *ETA priority*.

3.4. Bonus No. 2 Implementasi Hill-Climbing Tambahan

- Stochastic Hill Climbing

Stochastic Hill Climbing memilih secara acak salah satu neighbor yang memiliki objective lebih tinggi daripada state saat ini. Probabilitas pemilihan sebanding dengan besar peningkatannya

$$P(s' \mid s) = \frac{f(s') - f(s)}{\sum_{u \in N^+(s)} (f(u) - f(s))}$$

dengan $N^+(s)$ merupakan himpunan neighbor yang meningkatkan objective. Neighbor dengan peningkatan lebih besar memiliki peluang lebih tinggi, tetapi tidak selalu dipilih.

Pemilihan neighbor tersebut diimplementasikan menggunakan weighted reservoir sampling sebagai berikut:

```
if variant == "stochastic" and score > current_obj:
        delta = score - current_obj
        improvement_weight += delta
        if rng.randrange(improvement_weight) < delta:
            stochastic_neighbor, stochastic_score = neighbor, score
```

Variabel *improvement_weight* mengakumulasikan peningkatan objective. Setelah seluruh neighbor diperiksa, peluang setiap kandidat terpilih sebanding dengan besar peningkatannya.

- Hill Climbing with Sideways Move

  Varian ini tetap mengutamakan neighbor dengan objective tertinggi yang lebih baik. Apabila tidak ditemukan peningkatan, algoritma dapat berpindah ke neighbor berbeda dengan objective yang sama. Jika terdapat beberapa pilihan sama-skor, salah satunya dipilih secara acak. Perpindahan tersebut memungkinkan eksplorasi plateau. Parameter *max_sideways* membatasi jumlah sideways move berturut-turut. Hitungan direset setelah terjadi peningkatan. Pencarian berhenti jika tidak tersedia perpindahan yang diizinkan atau batas iterasi tercapai.

  Pemilihan kandidat dengan objective yang sama dilakukan secara acak:

```
if variant == "sideways" and score == current_obj:
        ties += 1
        if rng.randrange(ties) == 0:
            sideways_neighbor = neighbor
```

Kandidat tersebut hanya diterima jika tidak ditemukan peningkatan dan batas sideways berturut-turut belum tercapai:

```
if (
        selected is None
        and variant == "sideways"
        and sideways_neighbor is not None
        and sideways_streak < max_sideways
    ):
        selected, selected_score = sideways_neighbor, current_obj
        sideways_streak += 1
        sideways_moves += 1
    elif selected is not None:
        sideways_streak = 0
```

- Random Restart Hill Climbing

  Random Restart menjalankan pencarian Steepest Ascent dari beberapa initial state acak. Ketika suatu percobaan tidak lagi menemukan peningkatan, algoritma membentuk initial state baru dan mengulangi pencarian. Parameter *max_restart* membatasi jumlah restart di luar percobaan pertama. Batas *max_iterations* merupakan anggaran total seluruh percobaan, bukan batas terpisah untuk setiap restart. Algoritma menyimpan dan mengembalikan state terbaik dari seluruh percobaan, bukan hanya state terakhir. Jumlah restart dan iterasi setiap percobaan turut dicatat.

  Ketika tidak ditemukan peningkatan, algoritma memeriksa batas pencarian sebelum memulai restart:

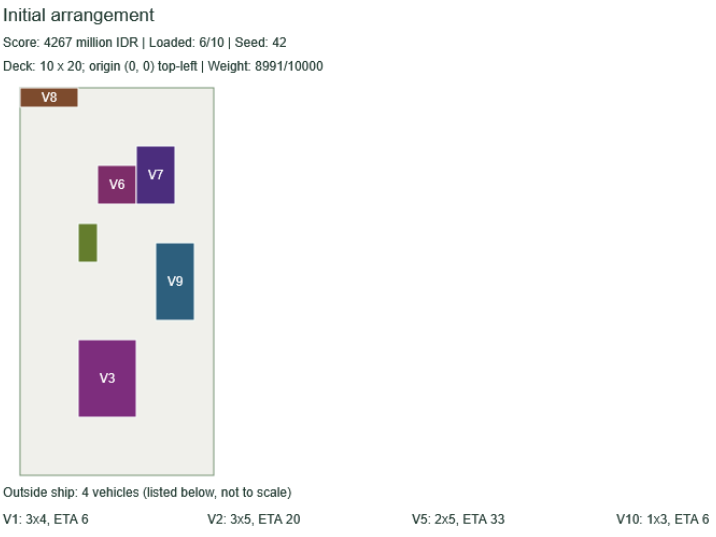
```
if selected is None:
        if variant == "random_restart":
            if max_iterations is not None and iterations >=
max_iterations:
                break
```

```
            if restarts >= max_restart:
                stop_reason = "restart_limit"
                break
            iterations_per_restart.append(attempt_iterations)
            attempt_iterations = 0
            restarts += 1
            initialization_started = perf_counter()
            current = initialize_arrangement(
                config, rng, preserve_positions=False,
should_cancel=should_cancel
            )
```

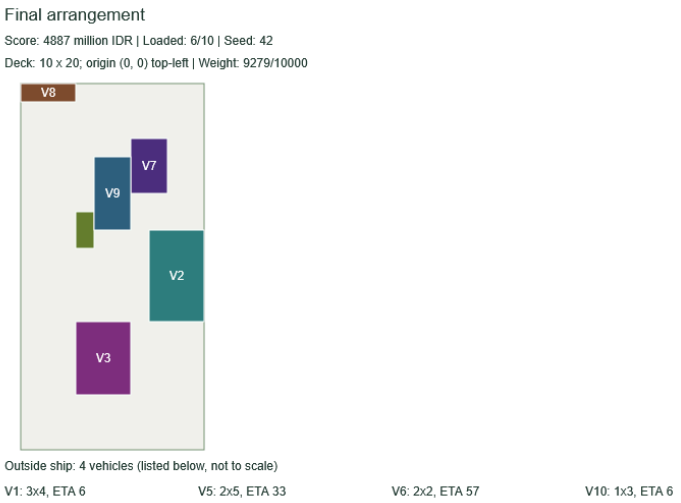
Setelah pencarian selesai, state terbaik dari seluruh percobaan digunakan sebagai hasil akhir:

```
iterations_per_restart.append(attempt_iterations)
    if variant == "random_restart":
        current, current_obj = best_arrangement, best_objective
```

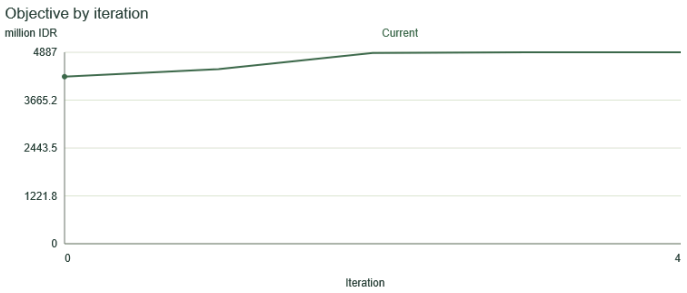
- Hasil Eksperimen Varian Hill Climbing
    - Stochastic
        - Initial State



        - Final State



- Objective

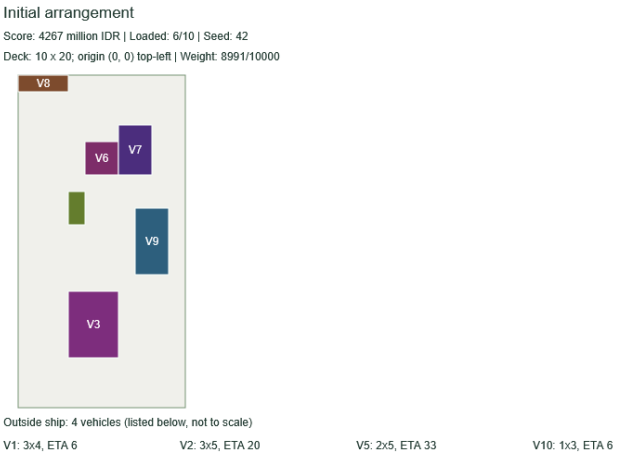


- Summary

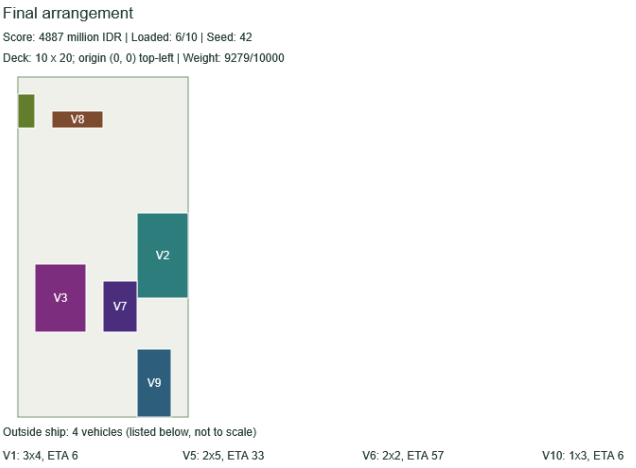

```
{
  "run_id":
"d26e18fd-28c9-4a5a-a350-9c45a20d9295",
  "created_at":
"2026-10-22T12:39:11.575147+00:00",
  "config_hash":
"d8fe3866175b0cfc6b54b3788051278199cfec4b50dd
a05aa592913724f20840",
  "random_seed": 42,
  "objective_mode": "shipping_fee",
  "objective_unit": "million IDR",
  "settings": {
    "algorithm": "hill_climbing",
    "iterations": 100,
    "variant": "stochastic",
    "max_sideways": 10,
    "max_restart": 3
  },
  "metrics": {
    "initial_objective": 4267,
```

```
  "final_objective": 4887,
  "runtime_ms": 532.6904239991563,
  "iterations_completed": 4,
  "neighbors_evaluated": 2302,
  "stop_reason": "local_optimum",
  "sideways_moves": 0,
  "restarts_completed": 0,
  "iterations_per_restart": [
   4
  ],
  "initial_loaded": 6,
  "final_loaded": 6,
  "final_unloaded": 4
 }
}
```


- Sideways Move
  - Initial State



  - Final State



- Objective

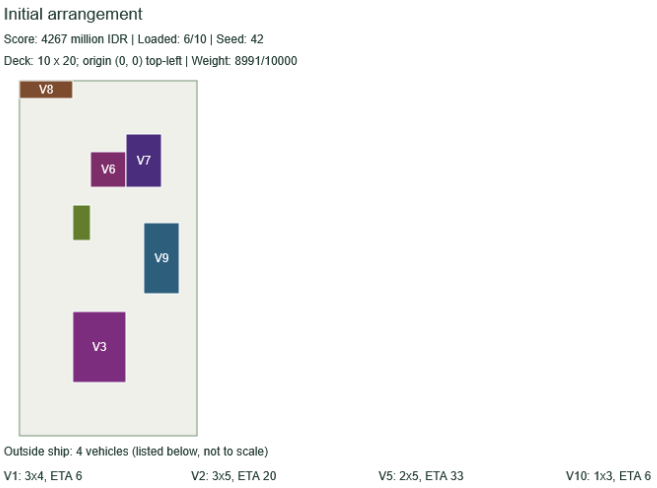


- Summary

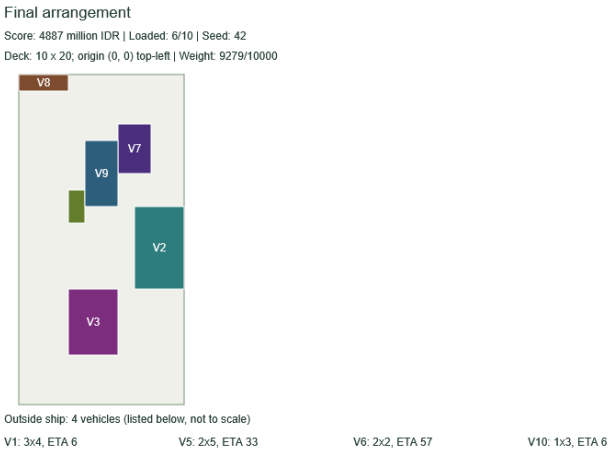

```
{
 "run_id":
"1ad8643b-c955-48f8-9d46-8df880433d26",
 "created_at":
"2026-10-22T12:39:14.815677+00:00",
 "config_hash":
"d8fe3866175b0cfc6b54b3788051278199cfec4b50dd
a05aa592913724f20840",
 "random_seed": 42,
 "objective_mode": "shipping_fee",
 "objective_unit": "million IDR",
 "settings": {
  "algorithm": "hill_climbing",
  "iterations": 100,
  "variant": "sideways",
  "max_sideways": 10,
  "max_restart": 3
 },
 "metrics": {
  "initial_objective": 4267,
  "final_objective": 4887,
  "runtime_ms": 1727.2263650011155,
```

```
    "iterations_completed": 14,
    "neighbors_evaluated": 7731,
    "stop_reason": "sideways_limit",
    "sideways_moves": 10,
    "restarts_completed": 0,
    "iterations_per_restart": [
      14
    ],
    "initial_loaded": 6,
    "final_loaded": 6,
    "final_unloaded": 4
  }
}
```

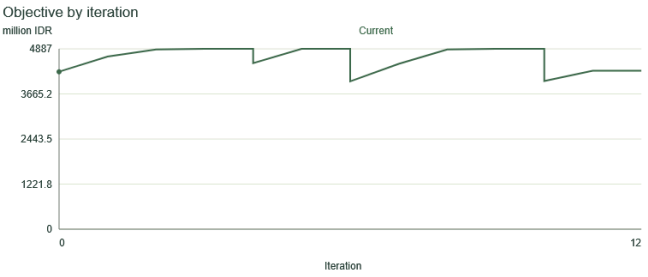

- Random Restart
  - Initial State



  - Final State



- Objective



- Summary


```
{
  "run_id":
"7d752221-42d5-4dcb-899d-8f45bcc018b0",
  "created_at":
"2026-10-22T12:39:19.835233+00:00",
  "config_hash":
"d8fe3866175b0cfc6b54b3788051278199cfec4b50dd
a05aa592913724f20840",
  "random_seed": 42,
  "objective_mode": "shipping_fee",
  "objective_unit": "million IDR",
  "settings": {
    "algorithm": "hill_climbing",
    "iterations": 100,
    "variant": "random_restart",
    "max_sideways": 10,
    "max_restart": 3
  },
  "metrics": {
    "initial_objective": 4267,
    "final_objective": 4887,
```

```
    "runtime_ms": 1402.990372000204,
    "iterations_completed": 12,
    "neighbors_evaluated": 6353,
    "stop_reason": "restart_limit",
    "sideways_moves": 0,
    "restarts_completed": 3,
    "iterations_per_restart": [
      4,
      2,
      4,
      2
    ],
    "initial_loaded": 6,
    "final_loaded": 6,
    "final_unloaded": 4
  }
}
```

BAB V PEMBAGIAN TUGAS

Nama	NIM	Tugas
Daniel Arrigo Manurung	18224031	Implementasi Algoritma Hill Climbing, Membuat laporan
Kenneth Moses Saragih	18224093	Implementasi Algoritma Simulated Annealing, Membuat laporan
Laurensius Dani Rendragraha	18224107	Implementasi Algoritma Genetic Algorithm, Membuat laporan

BAB VI REFERENSI

Chiao, A. (n.d.). Genetic algorithm visualization. GitHub. https://github.com/abelchiao/genetic-algorithm-visualization#algorithm-implementation

GeeksforGeeks. (n.d.). Genetic algorithms. https://www.geeksforgeeks.org/dsa/genetic-algorithms/

GeeksforGeeks. (2026, September 1). What is simulated annealing? https://www.geeksforgeeks.org/artificial-intelligence/what-is-simulated-annealing/

LAMPIRAN

1. Form Penggunaan Generative AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Daniel Arrigo Manurung/18224031; Kenneth Moses Saragih/18224093; Laurensius Dani Rendragraha/18224107
Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika
Kode Mata Kuliah : IF3070
Nama Mata Kuliah : Dasar Inteligensi Artifisial
Judul Tugas/Proposal : Optimasi Solusi Penataan Posisi dan Orientasi Kendaraan pada Kapal dengan Local Search

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	4
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	4
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Tidak	3
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	1
Lainnya, Jelaskan:	-	-

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026





Daniel Arrigo Manurung
18224031

Kenneth Moses Saragih
18224093

Laurensius Dani Rendragraha
18224107

Berikut adalah link formulir tersebut:

Form Pernyataan Penggunaan AI_IF3070_G13.pdf